Interactive Oracle Proofs of Proximity

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 IOPs of Proximity

2 Compiling Poly-IOPs using RS-IOPPs

3 Batching RS-IOPPs

4 Attempt #3: Putting Everything Together

What will we learn today?

- Recall: a **Poly-IOP** becomes a SNARK by (1) replacing polynomials with **polynomial commitments** and (2) applying **Fiat–Shamir**.

What will we learn today?

- Recall: a **Poly-IOP** becomes a SNARK by (1) replacing polynomials with **polynomial commitments** and (2) applying **Fiat–Shamir**.
- PlonK = Poly-IOP + KZG. Replacing KZG with Bulletproofs gives Halo2.

What will we learn today?

- Recall: a **Poly-IOP** becomes a SNARK by (1) replacing polynomials with **polynomial commitments** and (2) applying **Fiat–Shamir**.
- PlonK = Poly-IOP + KZG. Replacing KZG with Bulletproofs gives Halo2.
- Today: commitments that need only **hash functions**, based on Reed–Solomon codes.

What will we learn today?

- Recall: a **Poly-IOP** becomes a SNARK by (1) replacing polynomials with **polynomial commitments** and (2) applying **Fiat–Shamir**.
- PlonK = Poly-IOP + KZG. Replacing KZG with Bulletproofs gives Halo2.
- Today: commitments that need only **hash functions**, based on Reed–Solomon codes.

💬 *Key idea*

An RS-IOPP (Reed–Solomon IOP of Proximity) can be used as a polynomial commitment scheme, which turns a Poly-IOP into an IOP.

What will we learn today?

- Recall: a **Poly-IOP** becomes a SNARK by (1) replacing polynomials with **polynomial commitments** and (2) applying **Fiat–Shamir**.
- PlonK = Poly-IOP + KZG. Replacing KZG with Bulletproofs gives Halo2.
- Today: commitments that need only **hash functions**, based on Reed–Solomon codes.

💬 *Key idea*

An RS-IOPP (Reed–Solomon IOP of Proximity) can be used as a polynomial commitment scheme, which turns a Poly-IOP into an IOP.

💬 *Recall from the previous lecture*

$\mathrm{RS}[\mathbb{F}, \Omega, k] := \{\bar{f} : \Omega \to \mathbb{F},\ f \in \mathbb{F}[T]^{<k}\}$, rate $\rho = k/n$, and the list $\mathrm{List}[\mathbf{w}, C, \delta]$ of codewords δ-close to $\mathbf{w}$.

IOPs of Proximity

Interactive Oracle Proofs

Recall the idea of an **interactive oracle proof** (IOP):

- As in any interactive proof, $\mathcal{P}$ and $\mathcal{V}$ exchange messages over several rounds; the verifier's messages are random challenges.

Interactive Oracle Proofs

Recall the idea of an **interactive oracle proof** (IOP):

- As in any interactive proof, $\mathcal{P}$ and $\mathcal{V}$ exchange messages over several rounds; the verifier's messages are random challenges.
- The prover's messages are **oracles**: long strings that the verifier does not read in full, but only **queries** at a few positions.

Interactive Oracle Proofs

Recall the idea of an **interactive oracle proof** (IOP):

- As in any interactive proof, $\mathcal{P}$ and $\mathcal{V}$ exchange messages over several rounds; the verifier's messages are random challenges.
- The prover's messages are **oracles**: long strings that the verifier does not read in full, but only **queries** at a few positions.
- Hence the verifier runs in time much smaller than the total length of the prover's messages.

Interactive Oracle Proofs

Recall the idea of an **interactive oracle proof** (IOP):

- As in any interactive proof, $\mathcal{P}$ and $\mathcal{V}$ exchange messages over several rounds; the verifier's messages are random challenges.
- The prover's messages are **oracles**: long strings that the verifier does not read in full, but only **queries** at a few positions.
- Hence the verifier runs in time much smaller than the total length of the prover's messages.
- In practice, the prover commits to each oracle with a **Merkle tree**, and every query is answered with an opening.

Interactive Oracle Proofs

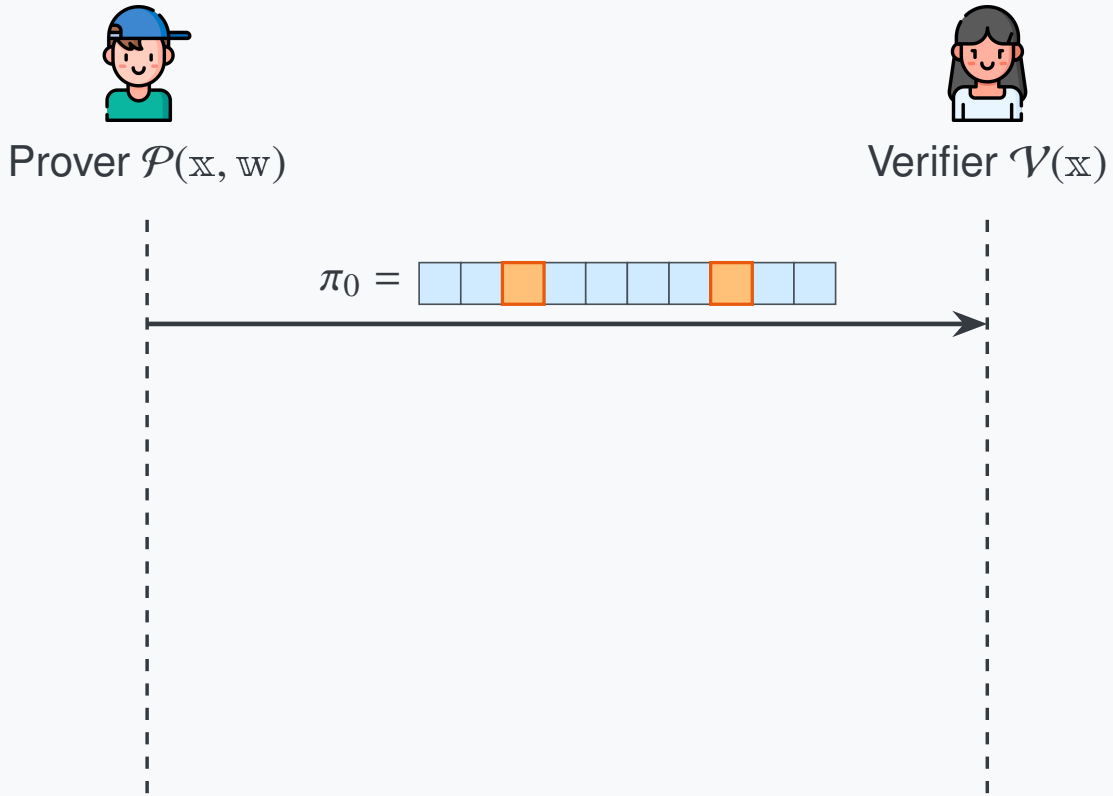


Figure: Long oracles π_i, short challenges α_i; the verifier queries a few cells (orange).

Interactive Oracle Proofs

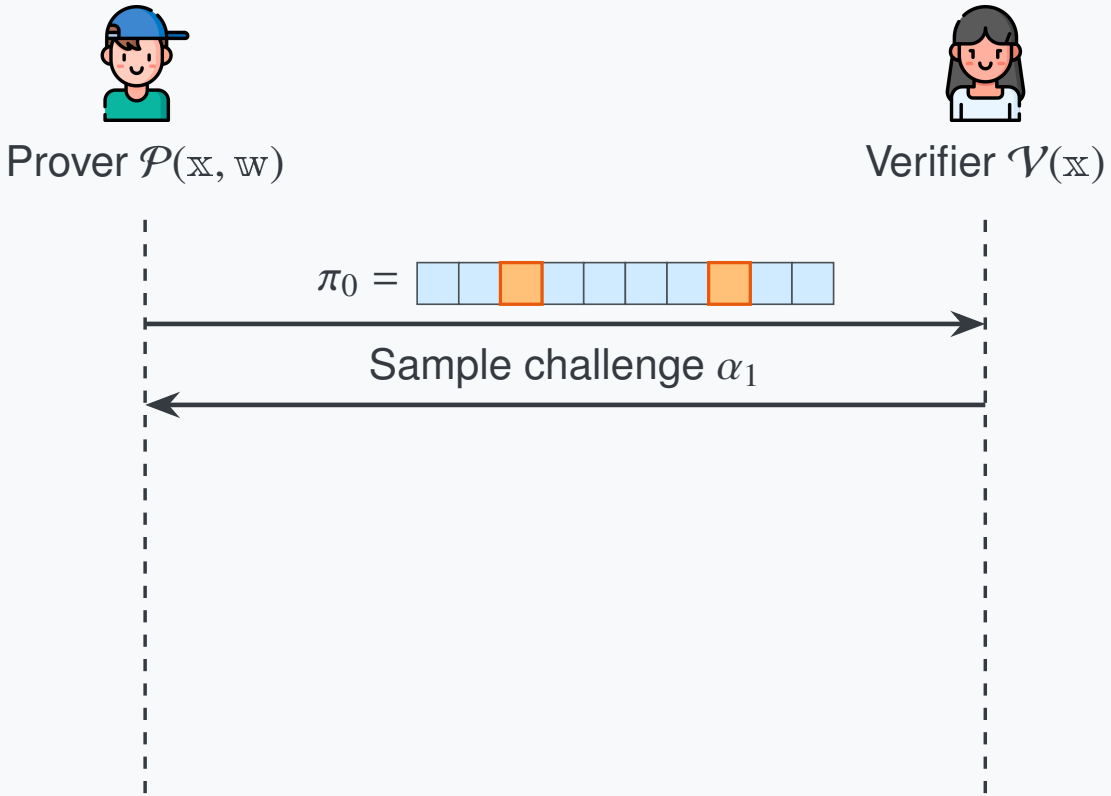


Figure: Long oracles π_i, short challenges α_i; the verifier queries a few cells (orange).

Interactive Oracle Proofs

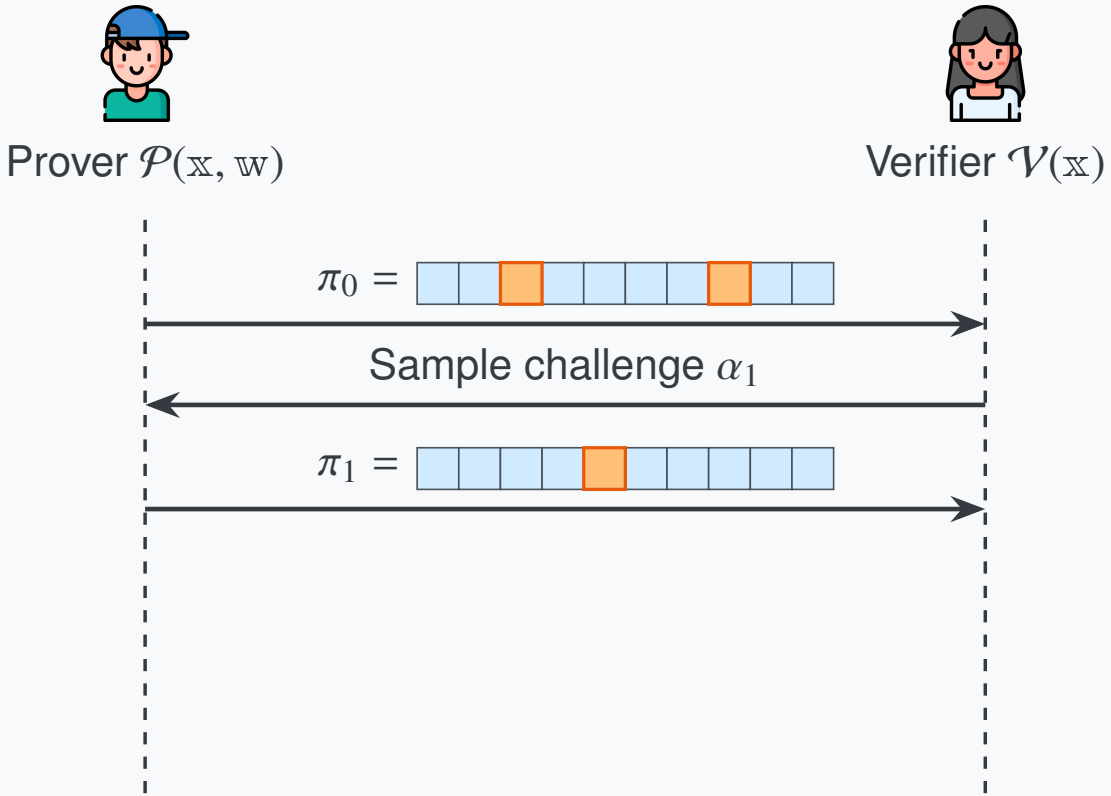


Figure: Long oracles π_i, short challenges α_i; the verifier queries a few cells (orange).

Interactive Oracle Proofs

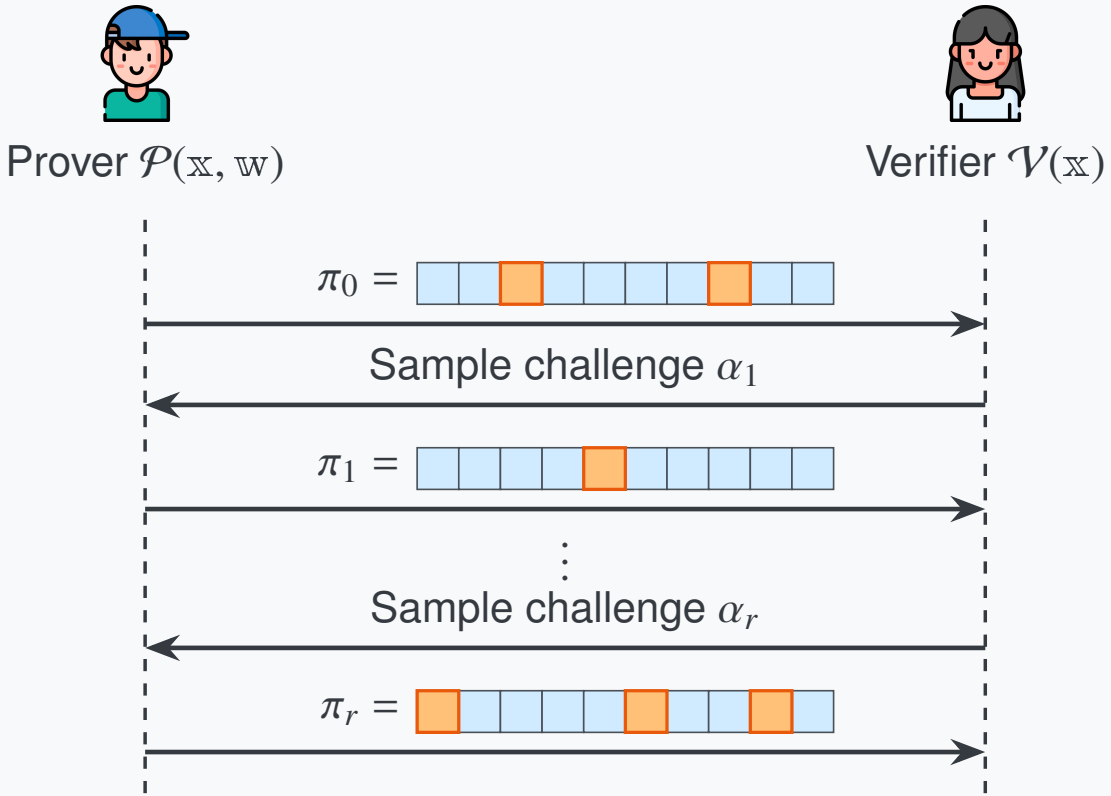


Figure: Long oracles π_i, short challenges α_i; the verifier queries a few cells (orange).

Interactive Oracle Proofs

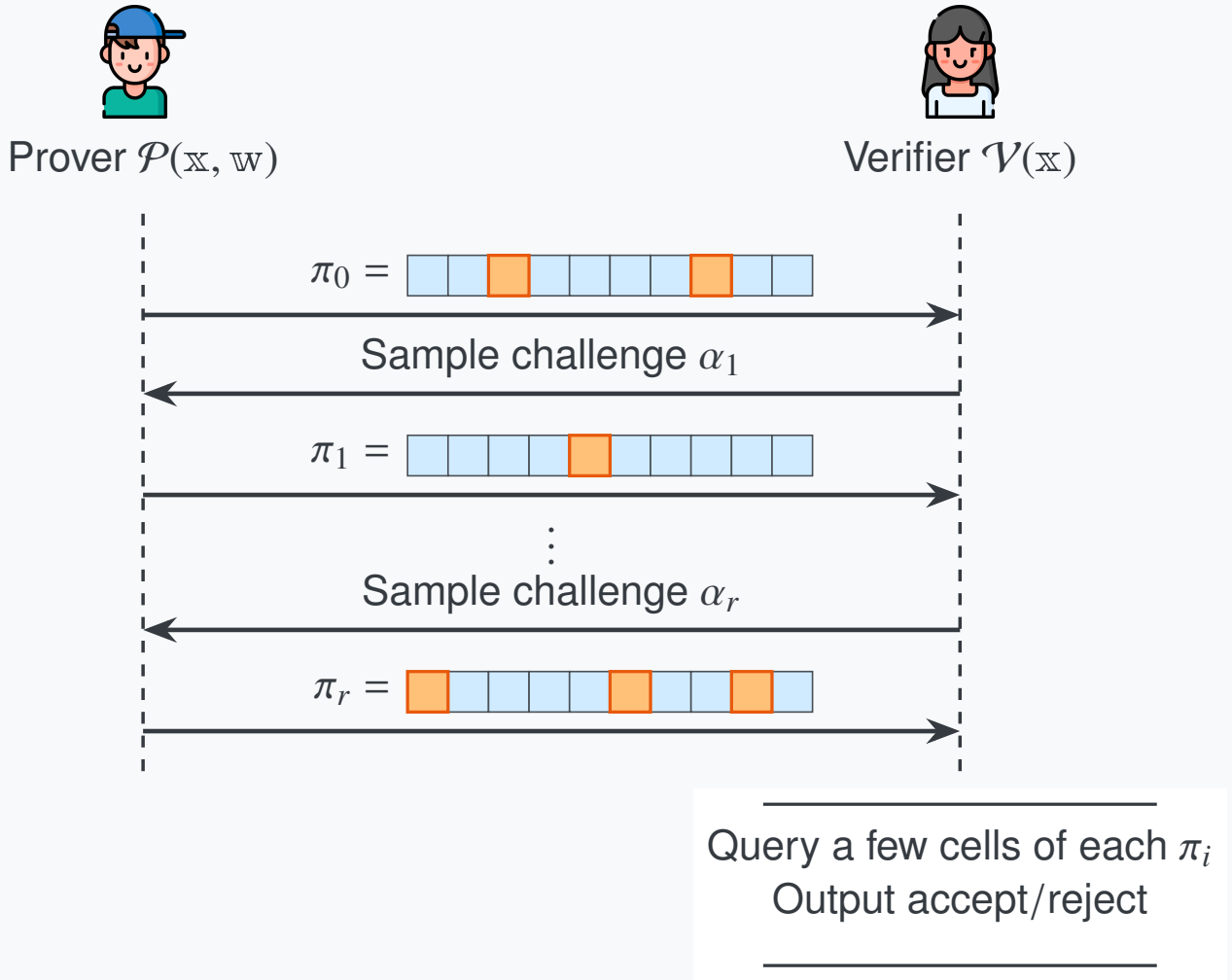


Figure: Long oracles π_i, short challenges α_i; the verifier queries a few cells (orange).

IOP of Proximity

In an IOP, the verifier reads the statement $\mathbb{x}$ in full. For the proximity problem this is not enough: the word $\mathbf{w}$ we want to check is itself too long to read.

IOP of Proximity

In an IOP, the verifier reads the statement $\mathbb{x}$ in full. For the proximity problem this is not enough: the word $\mathbf{w}$ we want to check is itself too long to read.

Definition (IOP of Proximity)

Let $\mathcal{R} = \{(\mathbb{x}, \mathbb{y}, \mathbb{w})\}$ be a relation, where $\mathbb{y}$ is a string of large length n. An **IOP of proximity** (IOPP) for $\mathcal{R}$ is an IOP in which

- the prover receives $(\mathbb{x}, \mathbb{y}, \mathbb{w})$;
- the verifier receives $\mathbb{x}$ and only **oracle access** to $\mathbb{y}$.

At the end, the verifier outputs

$$\mathcal{V}^{\mathbb{y},\pi_0,\ldots,\pi_r}(\mathbb{x}, \alpha_1, \ldots, \alpha_r) \to \{0, 1\}.$$

IOP of Proximity

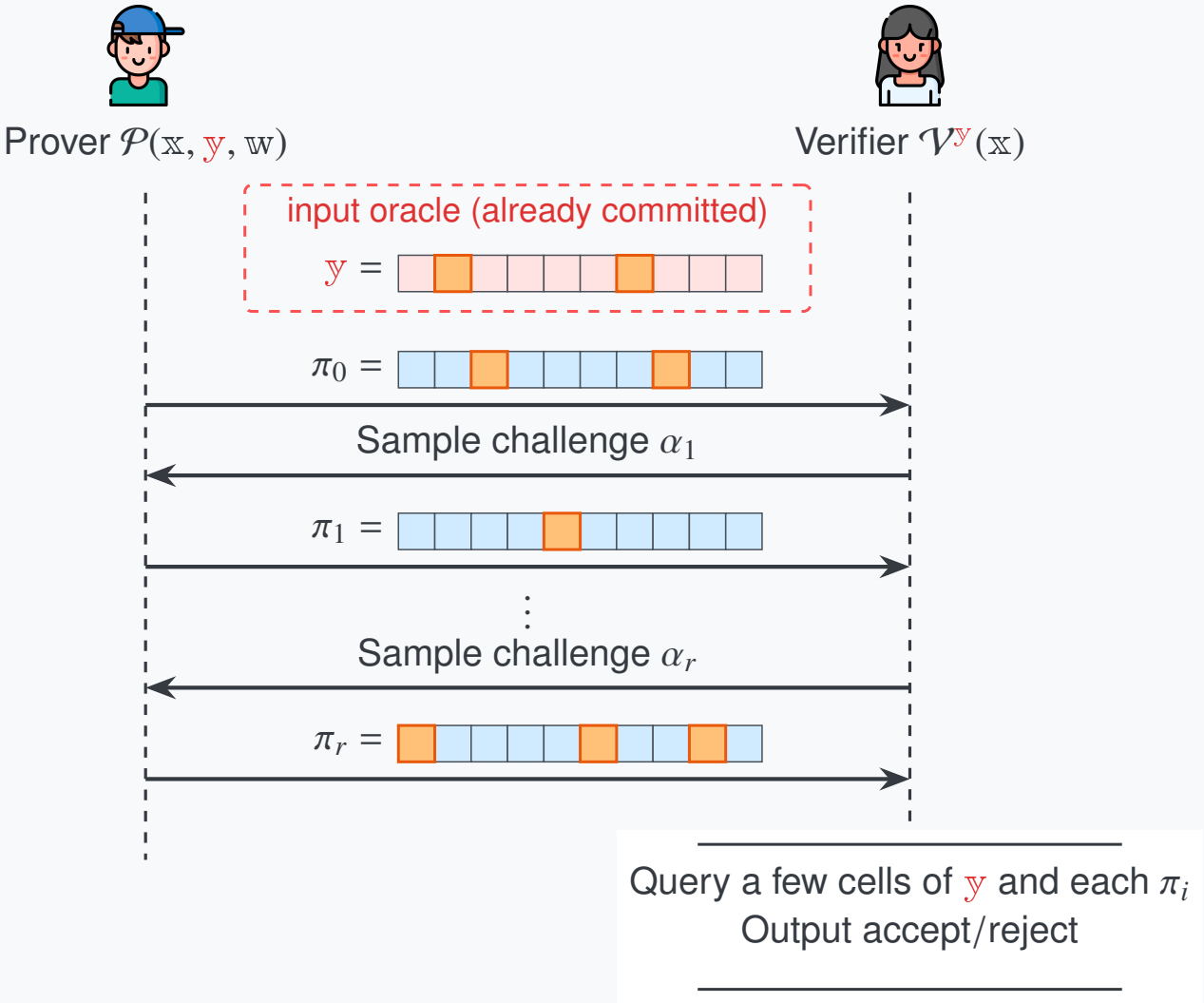


Figure: The input oracle $\mathbb{y}$: known to $\mathcal{P}$ in full, only queried by $\mathcal{V}$.

Which Inputs Should be Rejected?

The natural answer, “every $\mathbb{y}$ that is not valid”, is unachievable:

- take a valid $\mathbb{y}$ and change a single cell;

Which Inputs Should be Rejected?

The natural answer, “every $\mathbb{y}$ that is not valid”, is unachievable:

- take a valid $\mathbb{y}$ and change a single cell;
- a verifier querying q of the n cells reads the changed one with probability at most q/n.

Which Inputs Should be Rejected?

The natural answer, "every $\mathbb{y}$ that is not valid", is unachievable:

- take a valid $\mathbb{y}$ and change a single cell;
- a verifier querying q of the n cells reads the changed one with probability at most q/n.

We only ask to reject oracles that are far from every valid one.

Definition

$(\mathbb{x}, \mathbb{y})$ is δ**-far from** $\mathcal{R}$ if

$$(\mathbb{x}, \mathbb{y}', \mathbb{w}) \notin \mathcal{R} \text{ for all } \mathbb{y}', \mathbb{w} \text{ such that } \delta(\mathbb{y}, \mathbb{y}') \leq \delta.$$

Which Inputs Should be Rejected?

The natural answer, "every $\mathbb{y}$ that is not valid", is unachievable:

- take a valid $\mathbb{y}$ and change a single cell;
- a verifier querying q of the n cells reads the changed one with probability at most q/n.

We only ask to reject oracles that are far from every valid one.

Definition

$(\mathbb{x}, \mathbb{y})$ is δ**-far from** $\mathcal{R}$ if

$$(\mathbb{x}, \mathbb{y}', \mathbb{w}) \notin \mathcal{R} \text{ for all } \mathbb{y}', \mathbb{w} \text{ such that } \delta(\mathbb{y}, \mathbb{y}') \leq \delta.$$

That is, $\mathbb{y}$ cannot be made valid by changing at most a δ fraction of its cells.

Correctness and Soundness

Definition

An IOPP $(\mathcal{P}, \mathcal{V})$ for $\mathcal{R}$ may have the following properties:

- **Correctness.** For all $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$, $\mathcal{V}$ always accepts $\mathcal{P}$.
- δ**-soundness.** For all $(\mathbb{x}, \mathbb{y})$ that are δ-far from $\mathcal{R}$,

$$\forall_{\mathcal{P}^*} \ \Pr[\mathcal{V}^{\mathbb{y},\pi_0,\ldots,\pi_r}(\mathbb{x}, \alpha_1, \ldots, \alpha_r) = 1] < \mathsf{negl}(\lambda).$$

Correctness and Soundness

Definition

An IOPP $(\mathcal{P}, \mathcal{V})$ for $\mathcal{R}$ may have the following properties:

- **Correctness.** For all $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$, $\mathcal{V}$ always accepts $\mathcal{P}$.
- δ**-soundness.** For all $(\mathbb{x}, \mathbb{y})$ that are δ-far from $\mathcal{R}$,

$$\forall_{\mathcal{P}^*} \ \Pr[\mathcal{V}^{\mathbb{y},\pi_0,\ldots,\pi_r}(\mathbb{x}, \alpha_1, \ldots, \alpha_r) = 1] < \mathsf{negl}(\lambda).$$

The gap is harmless

An invalid $\mathbb{y}$ that is not δ-far may be accepted. But then $\mathbb{y}$ is δ-close to some valid $\mathbb{y}'$, which for a code and small δ is unique, or one of a bounded list (list decoding!). We treat the prover as committed to $\mathbb{y}'$.

Example

Example

Let $\mathbb{x} = \mathsf{RS}[\mathbb{F}_7, \Omega := \mathbb{F}_7, k := 2]$, and $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$ iff $\mathbb{w} = f \in \mathbb{F}_7[T]^{<2}$ and $\mathbb{y} = \bar{f}$. The valid oracles are exactly the codewords.

Example

Example

Let $\mathbb{x} = \mathsf{RS}[\mathbb{F}_7, \Omega := \mathbb{F}_7, k := 2]$, and $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$ iff $\mathbb{w} = f \in \mathbb{F}_7[T]^{<2}$ and $\mathbb{y} = \bar{f}$. The valid oracles are exactly the codewords.

- $\mathbb{y}_1 = (3, 5, 0, 2, 4, 6, 1)$ is the codeword of $f = 3 + 2T$: always accepted.

Example

Example

Let $\mathbb{x} = \mathsf{RS}[\mathbb{F}_7, \Omega := \mathbb{F}_7, k := 2]$, and $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$ iff $\mathbb{w} = f \in \mathbb{F}_7[T]^{<2}$ and $\mathbb{y} = \bar{f}$. The valid oracles are exactly the codewords.

- $\mathbb{y}_1 = (3, 5, 0, 2, 4, 6, 1)$ is the codeword of $f = 3 + 2T$: always accepted.
- $\mathbb{y}_2 = (3, 5, 0, 2, 4, 6, 0)$ is $1/7$-close to $\mathbb{y}_1$, so it is not δ-far for $\delta \geq 1/7$: may be accepted, and we regard the prover as committed to $f = 3 + 2T$.

Example

Example

Let $\mathbb{x} = \mathsf{RS}[\mathbb{F}_7, \Omega := \mathbb{F}_7, k := 2]$, and $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$ iff $\mathbb{w} = f \in \mathbb{F}_7[T]^{<2}$ and $\mathbb{y} = \bar{f}$. The valid oracles are exactly the codewords.

- $\mathbb{y}_1 = (3, 5, 0, 2, 4, 6, 1)$ is the codeword of $f = 3 + 2T$: always accepted.
- $\mathbb{y}_2 = (3, 5, 0, 2, 4, 6, 0)$ is $1/7$-close to $\mathbb{y}_1$, so it is not δ-far for $\delta \geq 1/7$: may be accepted, and we regard the prover as committed to $f = 3 + 2T$.
- $\mathbb{y}_3 = (0, 1, 4, 2, 2, 4, 1)$ are the evaluations of T^2: it differs from every codeword in at least 5 cells, so it is δ-far for every $\delta < 5/7$, and it is rejected with high probability.

Reed–Solomon IOPP

The special case where the statement is a code $C = \mathsf{RS}[\mathbb{F}, \Omega, k]$ and $\mathbb{y}$ is a restriction $g : \Omega \to \mathbb{F}$ is the central topic of this lecture.

Definition (Reed–Solomon IOPP)

Given oracle access to $g : \Omega \to \mathbb{F}$ and $\delta \in (0, 1)$, distinguish, with high confidence and few oracle queries $\pi_0, \ldots, \pi_r$, between

- the case $g \in \mathsf{RS}[\mathbb{F}, \Omega, k]$, and
- the case that g is δ-far from $\mathsf{RS}[\mathbb{F}, \Omega, k]$.

Reed–Solomon IOPP

The special case where the statement is a code $C = \mathsf{RS}[\mathbb{F}, \Omega, k]$ and $\mathbb{y}$ is a restriction $g : \Omega \to \mathbb{F}$ is the central topic of this lecture.

Definition (Reed–Solomon IOPP)

Given oracle access to $g : \Omega \to \mathbb{F}$ and $\delta \in (0, 1)$, distinguish, with high confidence and few oracle queries $\pi_0, \ldots, \pi_r$, between

- the case $g \in \mathsf{RS}[\mathbb{F}, \Omega, k]$, and
- the case that g is δ-far from $\mathsf{RS}[\mathbb{F}, \Omega, k]$.

Black box

Today we use a δ-RS-IOPP as a black box. Next lecture: how to build one (FRI).

Compiling Poly-IOPs using RS-IOPPs

Attempt #1: Encoding Polynomials

An IOP sends large **strings**, not **polynomials**. How do we encode $f \in \mathbb{F}[T]^{<k}$ into a string of length n?

Attempt #1: Encoding Polynomials

An IOP sends large **strings**, not **polynomials**. How do we encode $f \in \mathbb{F}[T]^{<k}$ into a string of length n?

- Natural answer: the restriction $\bar{f} : \Omega \to \mathbb{F}$, which lies in $\mathcal{C} := \mathsf{RS}[\mathbb{F}, \Omega, k]$.

Attempt #1: Encoding Polynomials

An IOP sends large **strings**, not **polynomials**. How do we encode $f \in \mathbb{F}[T]^{<k}$ into a string of length n?

- Natural answer: the restriction $\bar{f} : \Omega \to \mathbb{F}$, which lies in $\mathcal{C} := \mathsf{RS}[\mathbb{F}, \Omega, k]$.
- But the prover can send an arbitrary $g : \Omega \to \mathbb{F}$, so we need to verify that g is close to $\mathcal{C}$: exactly an RS-IOPP!

Attempt #1: Encoding Polynomials

An IOP sends large **strings**, not **polynomials**. How do we encode $f \in \mathbb{F}[T]^{<k}$ into a string of length n?

- Natural answer: the restriction $\bar{f} : \Omega \to \mathbb{F}$, which lies in $C := \mathsf{RS}[\mathbb{F}, \Omega, k]$.
- But the prover can send an arbitrary $g : \Omega \to \mathbb{F}$, so we need to verify that g is close to C: exactly an RS-IOPP!

That is not enough

A polynomial commitment needs **evaluation proofs**: that $f(r) = y$ for a given (r, y).

Quotienting

Proposition (Quotienting)

Let $r \in \mathbb{F} \setminus \Omega$, $y \in \mathbb{F}$, $f \in \mathbb{F}[T]^{<k}$, $\delta \in [0,1]$. Define the **quotient map** q sending a restriction $g : \Omega \to \mathbb{F}$ to the restriction $(g(T) - y)/(T - r)$. Then:

1. if $g = \bar{f} \in \mathsf{RS}[\mathbb{F}, \Omega, k]$ and $y = f(r)$, then $\mathsf{q}(g) \in \mathsf{RS}[\mathbb{F}, \Omega, k-1]$;
2. if $y \neq h(r)$ for all $\bar{h} \in \mathsf{List}[g, k, \delta]$, then $\mathsf{q}(g)$ is δ-far from $\mathsf{RS}[\mathbb{F}, \Omega, k-1]$.

Here, $\mathsf{List}[g, k, \delta] := \mathsf{List}[g, \mathsf{RS}[\mathbb{F}, \Omega, k], \delta]$. Since $r \notin \Omega$, $\mathsf{q}(g)$ is well-defined.

Quotienting

Proposition (Quotienting)

Let $r \in \mathbb{F} \setminus \Omega$, $y \in \mathbb{F}$, $f \in \mathbb{F}[T]^{<k}$, $\delta \in [0,1]$. Define the **quotient map** q sending a restriction $g : \Omega \to \mathbb{F}$ to the restriction $(g(T) - y)/(T - r)$. Then:

1. if $g = \overline{f} \in \mathrm{RS}[\mathbb{F}, \Omega, k]$ and $y = f(r)$, then $\mathrm{q}(g) \in \mathrm{RS}[\mathbb{F}, \Omega, k-1]$;
2. if $y \neq h(r)$ for all $\overline{h} \in \mathrm{List}[g, k, \delta]$, then $\mathrm{q}(g)$ is δ-far from $\mathrm{RS}[\mathbb{F}, \Omega, k-1]$.

Here, $\mathrm{List}[g, k, \delta] := \mathrm{List}[g, \mathrm{RS}[\mathbb{F}, \Omega, k], \delta]$. Since $r \notin \Omega$, $\mathrm{q}(g)$ is well-defined.

Proof idea.

(i) $f - y$ has a root at r, so $u := (f - y)/(T - r) \in \mathbb{F}[T]^{<k-1}$ and $\mathrm{q}(g) = \overline{u}$.

Quotienting

Proposition (Quotienting)

Let $r \in \mathbb{F} \setminus \Omega$, $y \in \mathbb{F}$, $f \in \mathbb{F}[T]^{<k}$, $\delta \in [0,1]$. Define the **quotient map** q sending a restriction $g : \Omega \to \mathbb{F}$ to the restriction $(g(T) - y)/(T - r)$. Then:

1. if $g = \overline{f} \in \mathrm{RS}[\mathbb{F}, \Omega, k]$ and $y = f(r)$, then $\mathrm{q}(g) \in \mathrm{RS}[\mathbb{F}, \Omega, k-1]$;
2. if $y \neq h(r)$ for all $\overline{h} \in \mathrm{List}[g, k, \delta]$, then $\mathrm{q}(g)$ is δ-far from $\mathrm{RS}[\mathbb{F}, \Omega, k-1]$.

Here, $\mathrm{List}[g, k, \delta] := \mathrm{List}[g, \mathrm{RS}[\mathbb{F}, \Omega, k], \delta]$. Since $r \notin \Omega$, $\mathrm{q}(g)$ is well-defined.

Proof idea.

(i) $f - y$ has a root at r, so $u := (f - y)/(T - r) \in \mathbb{F}[T]^{<k-1}$ and $\mathrm{q}(g) = \overline{u}$.

(ii) If $\mathrm{q}(g)$ is δ-close to some $\overline{u}$, then $h := u \cdot (T - r) + y$ satisfies $h(r) = y$ and $\overline{h} \in \mathrm{List}[g, k, \delta]$.

□

Quotienting: Example

Example

Let $\mathbb{F} = \mathbb{F}_7$, $\Omega = \mathbb{F}_7 \setminus \{0\}$, $k = 2$ and $r = 0 \notin \Omega$. Take $f = 3 + 2T$, so that $g = \overline{f} = (5, 0, 2, 4, 6, 1)$ and $f(r) = 3$.

Quotienting: Example

Example

Let $\mathbb{F} = \mathbb{F}_7$, $\Omega = \mathbb{F}_7 \setminus \{0\}$, $k = 2$ and $r = 0 \notin \Omega$. Take $f = 3 + 2T$, so that $g = \overline{f} = (5, 0, 2, 4, 6, 1)$ and $f(r) = 3$.

- ✓ Correct value $y = 3$:

$$\mathsf{q}(g)(\omega) = \frac{3 + 2\omega - 3}{\omega} = 2, \quad \mathsf{q}(g) = (2, 2, 2, 2, 2, 2) \in \mathsf{RS}[\mathbb{F}_7, \Omega, 1].$$

Quotienting: Example

Example

Let $\mathbb{F} = \mathbb{F}_7$, $\Omega = \mathbb{F}_7 \setminus \{0\}$, $k = 2$ and $r = 0 \notin \Omega$. Take $f = 3 + 2T$, so that $g = \bar{f} = (5, 0, 2, 4, 6, 1)$ and $f(r) = 3$.

- ✓ Correct value $y = 3$:

$$\mathrm{q}(g)(\omega) = \frac{3 + 2\omega - 3}{\omega} = 2, \quad \mathrm{q}(g) = (2, 2, 2, 2, 2, 2) \in \mathrm{RS}[\mathbb{F}_7, \Omega, 1].$$

- ✗ Wrong value $y = 4$: $\mathrm{q}(g)(\omega) = 2 - \omega^{-1}$, that is, $\mathrm{q}(g) = (1, 5, 4, 0, 6, 3)$. All six values are distinct, so it is at relative distance $5/6$ from the constants.

Quotienting: Example

Example

Let $\mathbb{F} = \mathbb{F}_7$, $\Omega = \mathbb{F}_7 \setminus \{0\}$, $k = 2$ and $r = 0 \notin \Omega$. Take $f = 3 + 2T$, so that $g = \bar{f} = (5, 0, 2, 4, 6, 1)$ and $f(r) = 3$.

- ✓ Correct value $y = 3$:

$$\mathrm{q}(g)(\omega) = \frac{3 + 2\omega - 3}{\omega} = 2, \quad \mathrm{q}(g) = (2, 2, 2, 2, 2, 2) \in \mathrm{RS}[\mathbb{F}_7, \Omega, 1].$$

- ✗ Wrong value $y = 4$: $\mathrm{q}(g)(\omega) = 2 - \omega^{-1}$, that is, $\mathrm{q}(g) = (1, 5, 4, 0, 6, 3)$. All six values are distinct, so it is at relative distance $5/6$ from the constants.

This matches the second claim: for small δ, $\mathrm{List}[g, 2, \delta] = \{\bar{f}\}$, and $f(0) = 3 \neq 4$.

Attempt #1

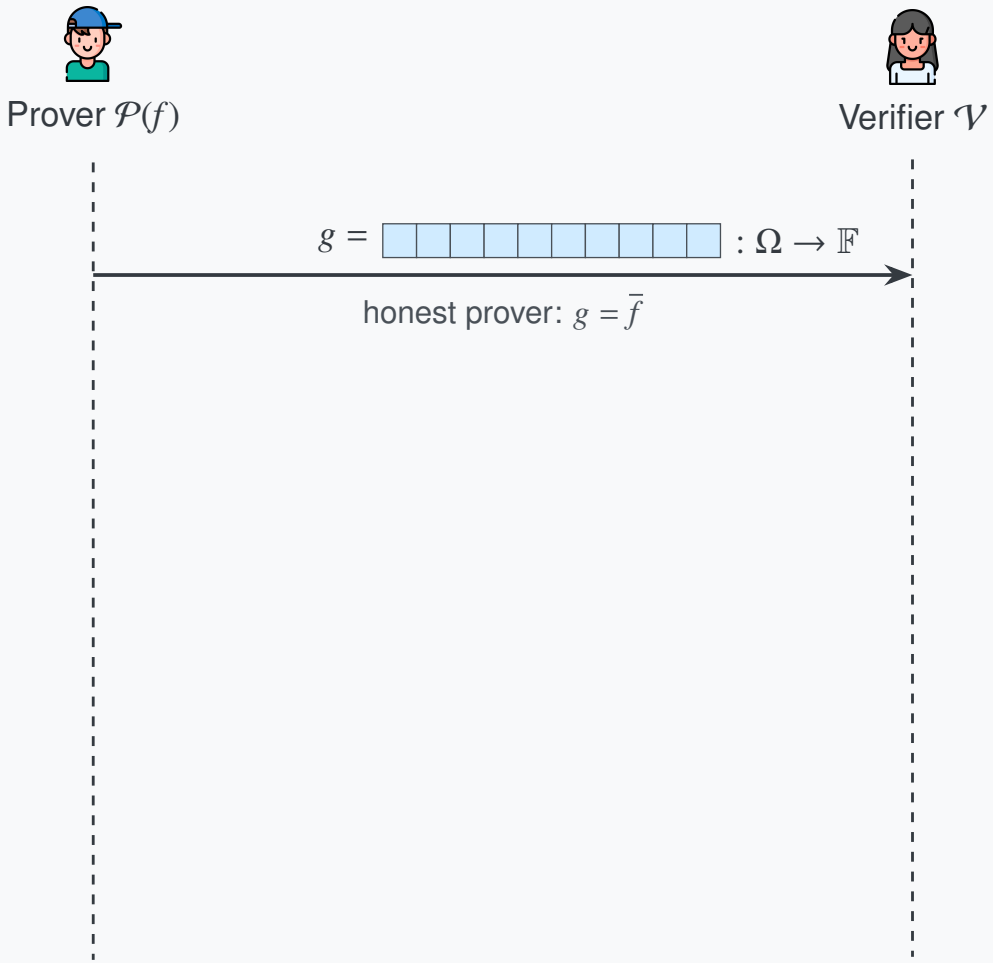


Figure: Virtual quotients $q_i(g) := (g - y_i)/(T - r_i)$: $\mathcal{V}$ computes their cells from g.

Attempt #1

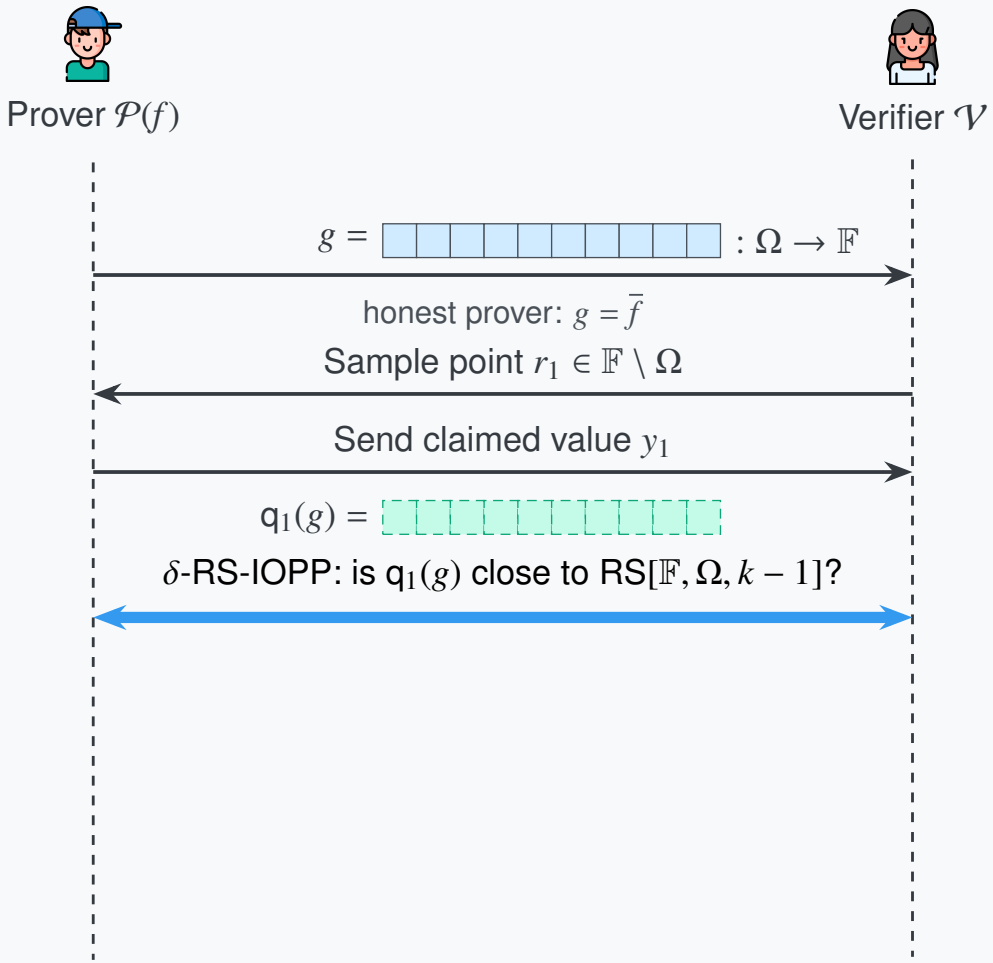


Figure: Virtual quotients $q_i(g) := (g - y_i)/(T - r_i)$: $\mathcal{V}$ computes their cells from g.

Attempt #1

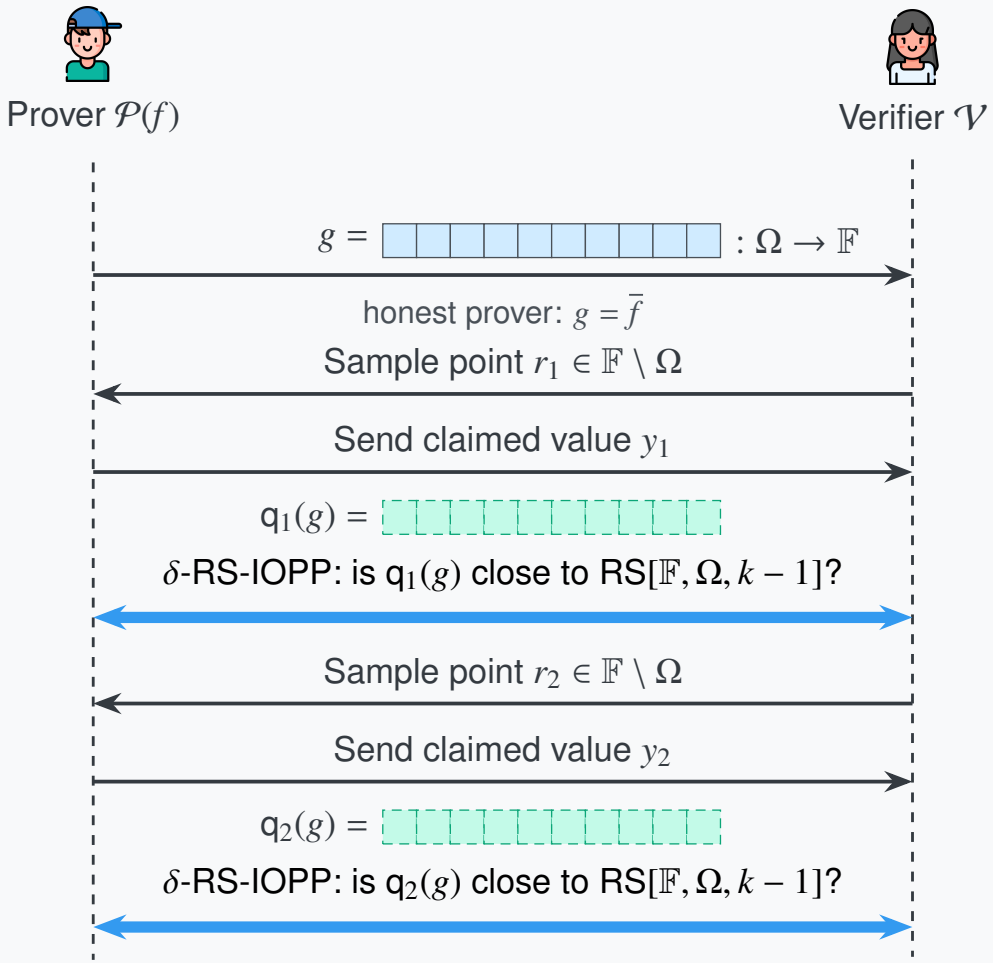


Figure: Virtual quotients $q_i(g) := (g - y_i)/(T - r_i)$: $\mathcal{V}$ computes their cells from g.

Attempt #1: What Goes Wrong?

What does the verifier $\mathcal{V}$ learn?

Attempt #1: What Goes Wrong?

What does the verifier $\mathcal{V}$ learn?

$\mathcal{V}$ learns

There are some $\bar{f}_1, \bar{f}_2 \in \text{List}[g, k, \delta]$ s.t. $f_1(r_1) = y_1$ and $f_2(r_2) = y_2$.

Attempt #1: What Goes Wrong?

What does the verifier $\mathcal{V}$ learn?

$\mathcal{V}$ learns

There are some $\bar{f}_1, \bar{f}_2 \in \mathsf{List}[g, k, \delta]$ s.t. $f_1(r_1) = y_1$ and $f_2(r_2) = y_2$.

Problem

For $\delta \geq \mu(C)/2$, the list $\mathsf{List}[g, k, \delta]$ may contain several elements, so it might happen that $f_1 \neq f_2$: different queries are explained by different codewords!

Attempt #2: Binding with a Random Point

Fix: before everything, $\mathcal{V}$ samples $r \leftarrow\!\!\$\ \mathbb{F} \setminus \Omega$, and an honest prover responds with $y \leftarrow f(r)$.

Attempt #2: Binding with a Random Point

Fix: before everything, $\mathcal{V}$ samples $r \leftarrow\$ \mathbb{F} \setminus \Omega$, and an honest prover responds with $y \leftarrow f(r)$.

Proposition

Let Bad be the event "there exist $\bar{f}_1 \neq \bar{f}_2 \in \mathsf{List}[g, k, \delta]$ such that $f_1(r) = f_2(r) = y$ for $r \leftarrow\$ \mathbb{F} \setminus \Omega$". Then

$$\Pr[\mathsf{Bad}] \leq \binom{\#\mathsf{List}[g, k, \delta]}{2} \cdot \frac{k-1}{\#\mathbb{F} - \#\Omega}.$$

Attempt #2: Binding with a Random Point

Fix: before everything, $\mathcal{V}$ samples $r \leftarrow\$ \mathbb{F} \setminus \Omega$, and an honest prover responds with $y \leftarrow f(r)$.

Proposition

Let Bad be the event "there exist $\bar{f}_1 \neq \bar{f}_2 \in \mathsf{List}[g, k, \delta]$ such that $f_1(r) = f_2(r) = y$ for $r \leftarrow\$ \mathbb{F} \setminus \Omega$". Then

$$\Pr[\mathsf{Bad}] \leq \binom{\#\mathsf{List}[g, k, \delta]}{2} \cdot \frac{k-1}{\#\mathbb{F} - \#\Omega}.$$

Proof. For a fixed pair, $f_1 - f_2 \neq 0$ has degree $< k$, so by Schwartz–Zippel $\Pr[f_1(r) = f_2(r)] \leq \frac{k-1}{\#\mathbb{F}-\#\Omega}$. Union bound over all pairs. □

Attempt #2: Probability of the Bad Event

Corollary

If $\delta < 1 - \sqrt{\rho(C)}$ (for $C = \mathsf{RS}[\mathbb{F}, \Omega, k]$), then

$$\Pr[\text{Bad}] \leq \frac{1}{2\varepsilon_\delta^2} \cdot \frac{k-1}{\#\mathbb{F} - \#\Omega}, \qquad \varepsilon_\delta = 2\sqrt{\rho(C)}\,(1 - \sqrt{\rho(C)} - \delta),$$

where ε_δ does not depend on the size of the field.

Proof. Johnson bound and $\binom{x}{2} \leq \frac{x^2}{2}$. □

Attempt #2: Probability of the Bad Event

Corollary

If $\delta < 1 - \sqrt{\rho(C)}$ (for $C = \mathsf{RS}[\mathbb{F}, \Omega, k]$), then

$$\Pr[\text{Bad}] \leq \frac{1}{2\varepsilon_\delta^2} \cdot \frac{k-1}{\#\mathbb{F} - \#\Omega}, \quad \varepsilon_\delta = 2\sqrt{\rho(C)}(1 - \sqrt{\rho(C)} - \delta),$$

where ε_δ does not depend on the size of the field.

Proof. Johnson bound and $\binom{x}{2} \leq \frac{x^2}{2}$. □

Remark

The probability is roughly const/$\#\mathbb{F}$: negligible for large fields. For small fields (like $\mathtt{M31}$, $\#\mathbb{F} = 2^{31} - 1$), repeat the procedure several times.

Attempt #2: Probability of the Bad Event

Corollary

If $\delta < 1 - \sqrt{\rho(C)}$ (for $C = \mathsf{RS}[\mathbb{F}, \Omega, k]$), then

$$\Pr[\text{Bad}] \leq \frac{1}{2\varepsilon_\delta^2} \cdot \frac{k-1}{\#\mathbb{F} - \#\Omega}, \quad \varepsilon_\delta = 2\sqrt{\rho(C)}(1 - \sqrt{\rho(C)} - \delta),$$

where ε_δ does not depend on the size of the field.

Proof. Johnson bound and $\binom{x}{2} \leq \frac{x^2}{2}$. □

Remark

The probability is roughly const/$\#\mathbb{F}$: negligible for large fields. For small fields (like `M31`, $\#\mathbb{F} = 2^{31} - 1$), repeat the procedure several times.

Hence (r, y) binds the prover to **only one** $\bar{f} \in \mathsf{List}[g, k, \delta]$. Remaining problem: aggregate the checks $f(r) = y$ and $f(r_i) = y_i$.

Multiple Quotientings

Proposition (Multiple Quotientings)

Let $(r_i)_{i\in[t]} \subseteq \mathbb{F} \setminus \Omega$, $(y_i)_{i\in[t]} \subseteq \mathbb{F}$, and $\delta \in [0,1]$. Define $\nu, \iota \in \mathbb{F}[T]^{\leq t}$:

$$\nu(T) := \prod_{i\in[t]}(T - r_i), \quad \iota(r_i) = y_i \text{ for all } i \in [t],$$

and the **quotient map** $\mathsf{q} : g \mapsto (g(T) - \iota(T))/\nu(T)$. Then:

1. if $g = \bar{f} \in \mathsf{RS}[\mathbb{F}, \Omega, k]$ and $y_i = f(r_i)$ for all $i \in [t]$, then $\mathsf{q}(g) \in \mathsf{RS}[\mathbb{F}, \Omega, k - t]$;
2. if for every $\bar{h} \in \mathsf{List}[g, k, \delta]$ we have $y_i \neq h(r_i)$ for some $i \in [t]$, then $\mathsf{q}(g)$ is δ-far from $\mathsf{RS}[\mathbb{F}, \Omega, k - t]$.

Multiple Quotientings

Proposition (Multiple Quotientings)

Let $(r_i)_{i\in[t]} \subseteq \mathbb{F} \setminus \Omega$, $(y_i)_{i\in[t]} \subseteq \mathbb{F}$, and $\delta \in [0,1]$. Define $\nu, \iota \in \mathbb{F}[T]^{\leq t}$:

$$\nu(T) := \prod_{i\in[t]}(T - r_i), \quad \iota(r_i) = y_i \text{ for all } i \in [t],$$

and the **quotient map** $\mathsf{q} : g \mapsto (g(T) - \iota(T))/\nu(T)$. Then:

1. if $g = \bar{f} \in \mathsf{RS}[\mathbb{F}, \Omega, k]$ and $y_i = f(r_i)$ for all $i \in [t]$, then $\mathsf{q}(g) \in \mathsf{RS}[\mathbb{F}, \Omega, k - t]$;
2. if for every $\bar{h} \in \mathsf{List}[g, k, \delta]$ we have $y_i \neq h(r_i)$ for some $i \in [t]$, then $\mathsf{q}(g)$ is δ-far from $\mathsf{RS}[\mathbb{F}, \Omega, k - t]$.

Question #1

Prove this fact (generalize the proof of single quotienting).

Attempt #2

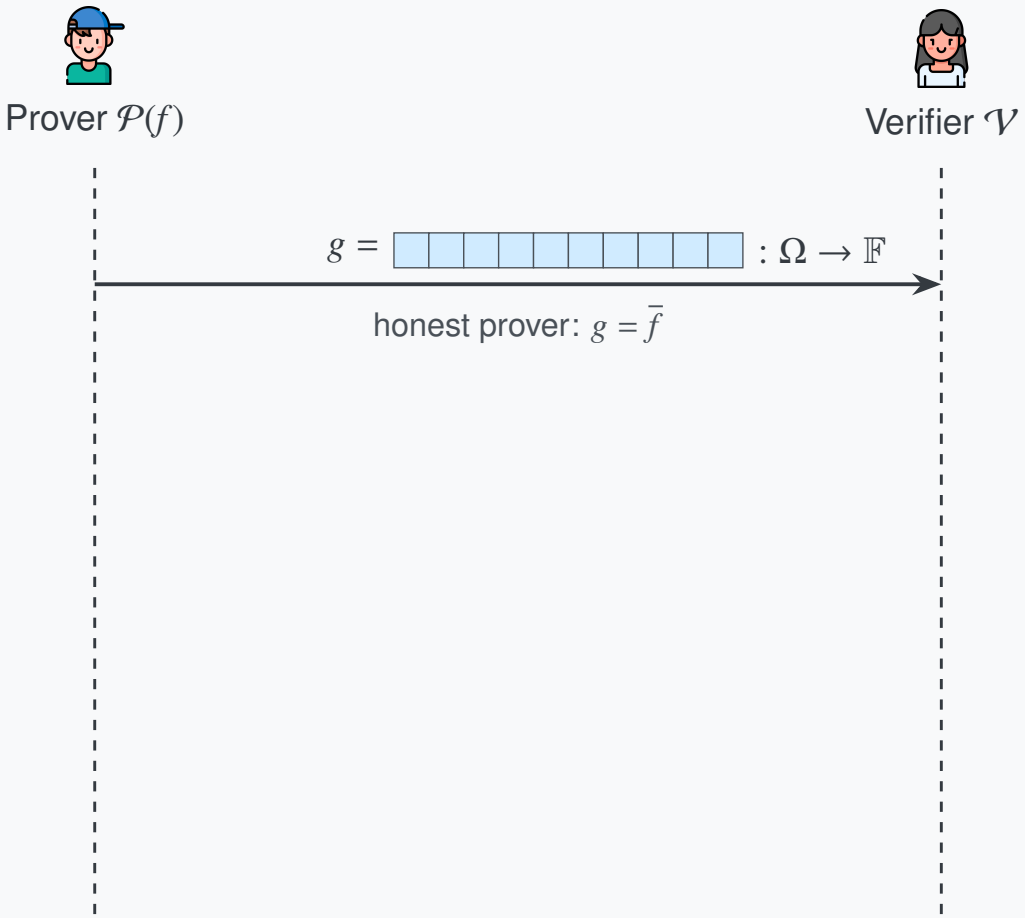


Figure: New: a random out-of-domain point r (highlighted); later queries are quotiented *together* with (r, y).

Attempt #2

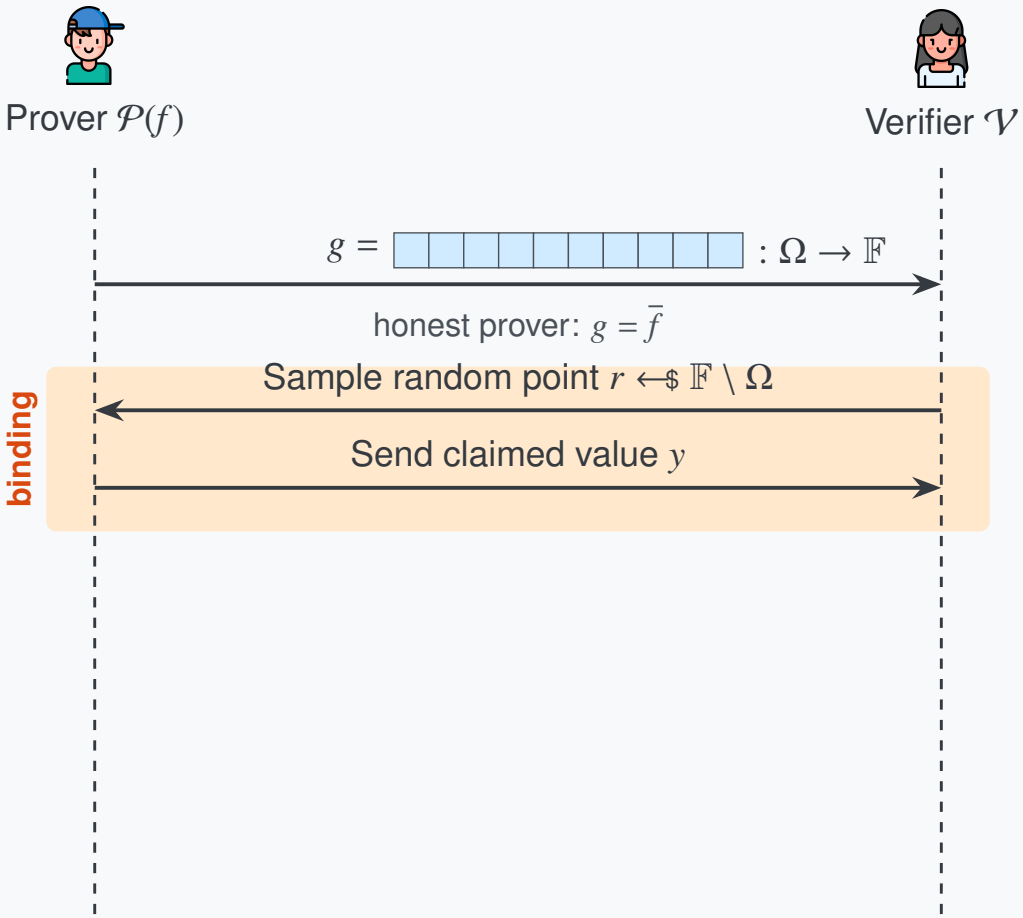


Figure: New: a random out-of-domain point r (highlighted); later queries are quotiented *together* with (r, y).

Attempt #2

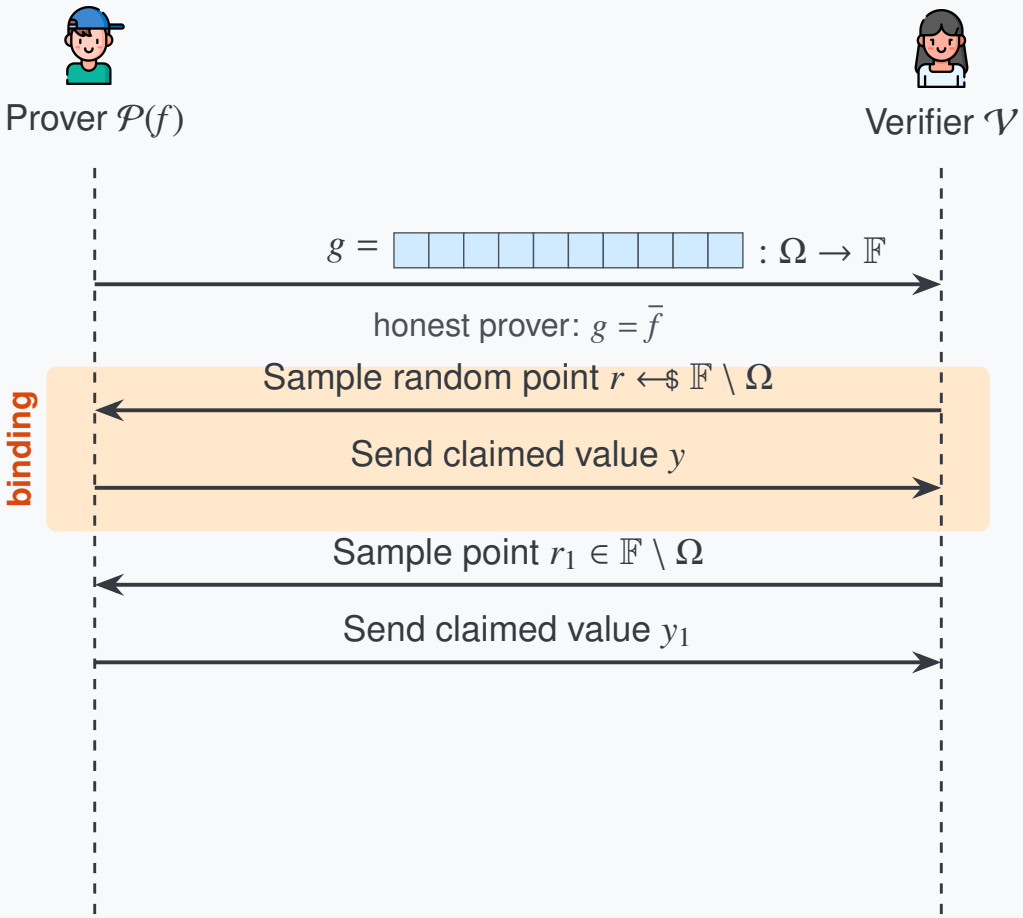


Figure: New: a random out-of-domain point r (highlighted); later queries are quotiented *together* with (r, y).

Attempt #2

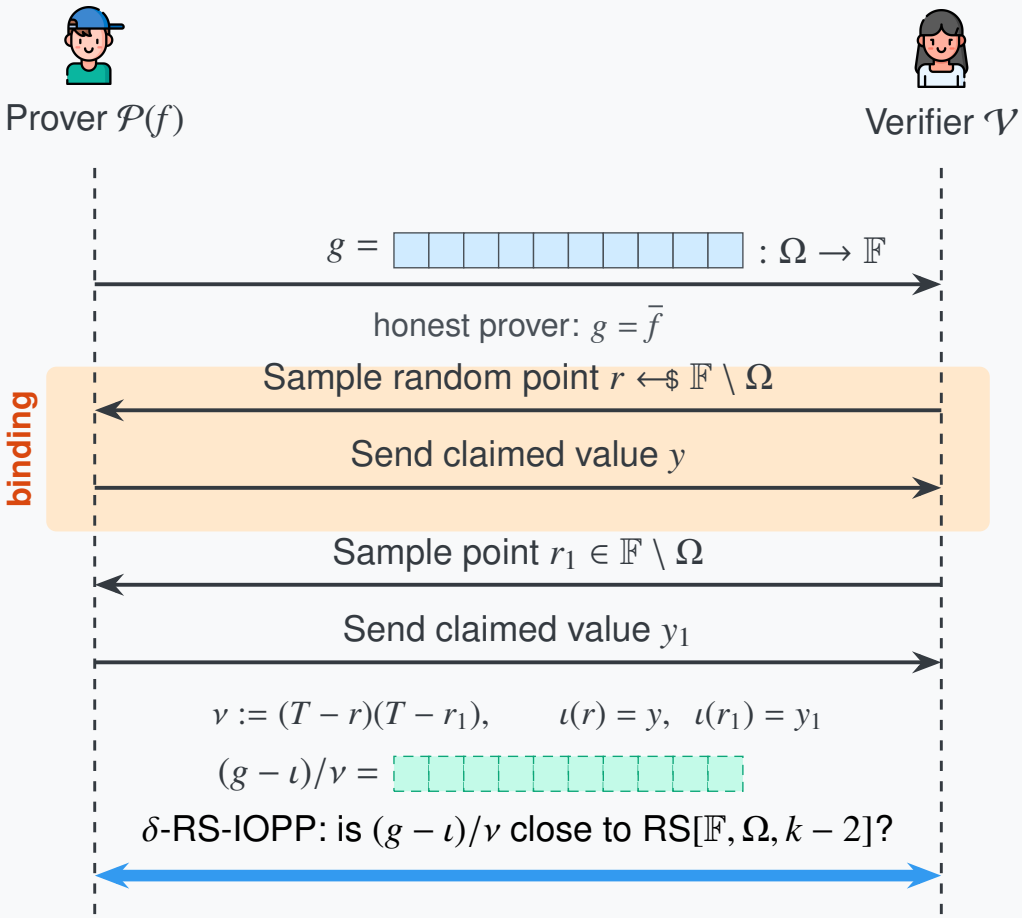


Figure: New: a random out-of-domain point r (highlighted); later queries are quotiented *together* with (r, y).

Batching RS-IOPPs

Batching RS-IOPPs

Running one RS-IOPP per query is costly. Given oracles $g_0, \ldots, g_n : \Omega \to \mathbb{F}$, the prover wants to convince $\mathcal{V}$ that all of them are δ-close to RS$[\mathbb{F}, \Omega, k]$.

Batching RS-IOPPs

Running one RS-IOPP per query is costly. Given oracles $g_0, \ldots, g_n : \Omega \to \mathbb{F}$, the prover wants to convince $\mathcal{V}$ that all of them are δ-close to $\mathrm{RS}[\mathbb{F}, \Omega, k]$.

Definition (Distance Preserving Transformation)

Let $\Omega, \Omega' \subseteq \mathbb{F}$ be evaluation domains and r a blinding factor. The map $T : (g_0, \ldots, g_n; r) \mapsto g$, from restrictions $g_i : \Omega \to \mathbb{F}$ to $g : \Omega' \to \mathbb{F}$, is **distance preserving** if:

1. if $g_0, \ldots, g_n \in \mathrm{RS}[\mathbb{F}, \Omega, k]$, then $g \in \mathrm{RS}[\mathbb{F}, \Omega', k']$ for all r;
2. if some g_i is δ-far from $\mathrm{RS}[\mathbb{F}, \Omega, k]$, then g is δ-far from $\mathrm{RS}[\mathbb{F}, \Omega', k']$ with overwhelming probability over r.

Linear Combination

Example

For r sampled randomly from $\mathbb{F}$,

$$T(g_0, \ldots, g_n; r) := g_0 + g_1 r + g_2 r^2 + \cdots + g_n r^n$$

is a distance preserving transformation (for appropriate code parameters).

Linear Combination

Example

For r sampled randomly from $\mathbb{F}$,

$$T(g_0, \ldots, g_n; r) := g_0 + g_1 r + g_2 r^2 + \cdots + g_n r^n$$

is a distance preserving transformation (for appropriate code parameters).

Batching: $\mathcal{V}$ samples $r \leftarrow\$ \mathbb{F}$ and the prover sets $g := \sum_i r^i g_i$. Then one RS-IOPP for g suffices.

Linear Combination

Example

For r sampled randomly from $\mathbb{F}$,

$$T(g_0, \ldots, g_n; r) := g_0 + g_1 r + g_2 r^2 + \cdots + g_n r^n$$

is a distance preserving transformation (for appropriate code parameters).

Batching: $\mathcal{V}$ samples $r \leftarrow\$ \mathbb{F}$ and the prover sets $g := \sum_i r^i g_i$. Then one RS-IOPP for g suffices.

- Property 1 is immediate: a linear combination of codewords is a codeword.

Linear Combination

Example

For r sampled randomly from $\mathbb{F}$,

$$T(g_0, \ldots, g_n; r) := g_0 + g_1 r + g_2 r^2 + \cdots + g_n r^n$$

is a distance preserving transformation (for appropriate code parameters).

Batching: $\mathcal{V}$ samples $r \leftarrow\$ \mathbb{F}$ and the prover sets $g := \sum_i r^i g_i$. Then one RS-IOPP for g suffices.

- Property 1 is immediate: a linear combination of codewords is a codeword.
- Property 2 is tricky: it follows from the **proximity gap theorem**.

Proximity Gaps

Lemma (The proximity gap theorem, simplified)

Let $C := \mathsf{RS}[\mathbb{F}, \Omega, k]$, $g_0, \ldots, g_n : \Omega \to \mathbb{F}$, $\delta < 1 - 1.01\sqrt{\rho}$ and $g_r := \sum_i r^i g_i$. Suppose

$$\Pr_{r \leftarrow \$ \mathbb{F}} [\delta(g_r, C) \leq \delta] > \varepsilon_\delta,$$

where $\varepsilon_\delta = O(\frac{nk}{\rho \# \mathbb{F}})$ for $\delta < \frac{1-\rho}{2}$ and $\varepsilon_\delta = O(\frac{nk^2}{\rho^2 \# \mathbb{F}})$ otherwise. Then all g_i are δ-close to C.

Proximity Gaps

Lemma (The proximity gap theorem, simplified)

Let $C := \mathrm{RS}[\mathbb{F}, \Omega, k]$, $g_0, \ldots, g_n : \Omega \to \mathbb{F}$, $\delta < 1 - 1.01\sqrt{\rho}$ and $g_r := \sum_i r^i g_i$. Suppose

$$\Pr_{r \leftarrow \$ \mathbb{F}} [\delta(g_r, C) \leq \delta] > \varepsilon_\delta,$$

where $\varepsilon_\delta = O(\frac{nk}{\rho \# \mathbb{F}})$ for $\delta < \frac{1-\rho}{2}$ and $\varepsilon_\delta = O(\frac{nk^2}{\rho^2 \# \mathbb{F}})$ otherwise. Then all g_i are δ-close to C.

Theorem (BCIKS20)

Moreover, there exists $S \subseteq \Omega$ such that $\#S \geq (1 - \delta)\#\Omega$ and

$$\forall_i \; \exists_{f_i \in C} \quad g_i\big|_S = f_i\big|_S.$$

Proximity Gaps

Lemma (The proximity gap theorem, simplified)

Let $C := \mathsf{RS}[\mathbb{F}, \Omega, k]$, $g_0, \ldots, g_n : \Omega \to \mathbb{F}$, $\delta < 1 - 1.01\sqrt{\rho}$ and $g_r := \sum_i r^i g_i$. Suppose

$$\Pr_{r \leftarrow \$ \mathbb{F}} [\delta(g_r, C) \leq \delta] > \varepsilon_\delta,$$

where $\varepsilon_\delta = O(\frac{nk}{\rho \# \mathbb{F}})$ for $\delta < \frac{1-\rho}{2}$ and $\varepsilon_\delta = O(\frac{nk^2}{\rho^2 \# \mathbb{F}})$ otherwise. Then all g_i are δ-close to C.

Theorem (BCIKS20)

Moreover, there exists $S \subseteq \Omega$ such that $\#S \geq (1-\delta)\#\Omega$ and

$$\forall_i \; \exists_{f_i \in C} \quad g_i\big|_S = f_i\big|_S.$$

That is, all g_i are δ-close to C **on the same positions** S.

2-way Folding: Splitting a Polynomial

Another distance preserving transformation shrinks the **code** instead of the number of functions.

2-way Folding: Splitting a Polynomial

Another distance preserving transformation shrinks the **code** instead of the number of functions.

- For $f \in \mathbb{F}[T]^{<k}$, let f_E (f_O) consist of the even (odd) coefficients of f:

$$f(T) = f_E(T^2) + Tf_O(T^2), \quad f_E, f_O \in \mathbb{F}[T]^{<k/2}.$$

2-way Folding: Splitting a Polynomial

Another distance preserving transformation shrinks the **code** instead of the number of functions.

- For $f \in \mathbb{F}[T]^{<k}$, let f_E (f_O) consist of the even (odd) coefficients of f:

$$f(T) = f_E(T^2) + Tf_O(T^2), \quad f_E, f_O \in \mathbb{F}[T]^{<k/2}.$$

- Let $\Omega := \langle \omega \rangle$, where ω is a primitive nth root of unity, n a power of two.

2-way Folding: Splitting a Polynomial

Another distance preserving transformation shrinks the **code** instead of the number of functions.

- For $f \in \mathbb{F}[T]^{<k}$, let f_E (f_O) consist of the even (odd) coefficients of f:

$$f(T) = f_E(T^2) + Tf_O(T^2), \quad f_E, f_O \in \mathbb{F}[T]^{<k/2}.$$

- Let $\Omega := \langle \omega \rangle$, where ω is a primitive nth root of unity, n a power of two.
- Since $\omega^{n/2} = -1$, Ω splits into pairs $\{x, -x\}$, and both elements of a pair are mapped to the same element of $\Omega^2 := \{x^2 : x \in \Omega\}$.

2-way Folding: Splitting a Polynomial

Another distance preserving transformation shrinks the **code** instead of the number of functions.

- For $f \in \mathbb{F}[T]^{<k}$, let f_E (f_O) consist of the even (odd) coefficients of f:

$$f(T) = f_E(T^2) + Tf_O(T^2), \quad f_E, f_O \in \mathbb{F}[T]^{<k/2}.$$

- Let $\Omega := \langle\omega\rangle$, where ω is a primitive nth root of unity, n a power of two.
- Since $\omega^{n/2} = -1$, Ω splits into pairs $\{x, -x\}$, and both elements of a pair are mapped to the same element of $\Omega^2 := \{x^2 : x \in \Omega\}$.

Question #2

Verify that $\#\Omega^2 = \frac{1}{2}\#\Omega$, so the evaluation domain shrinks $2\times$.

Squaring Halves a Smooth Domain

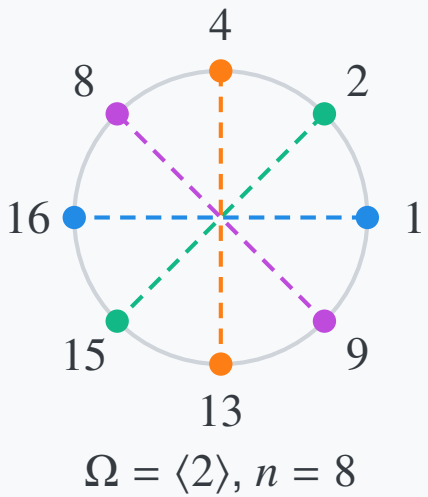


Figure: In $\mathbb{F}_{17}$: in $\Omega = \langle 2 \rangle$, every x has $-x$ on the opposite side (dashed), and both are mapped to the point of the same colour in Ω^2. Applying the map again gives $\Omega^4 = \{1, -1\}$.

Squaring Halves a Smooth Domain

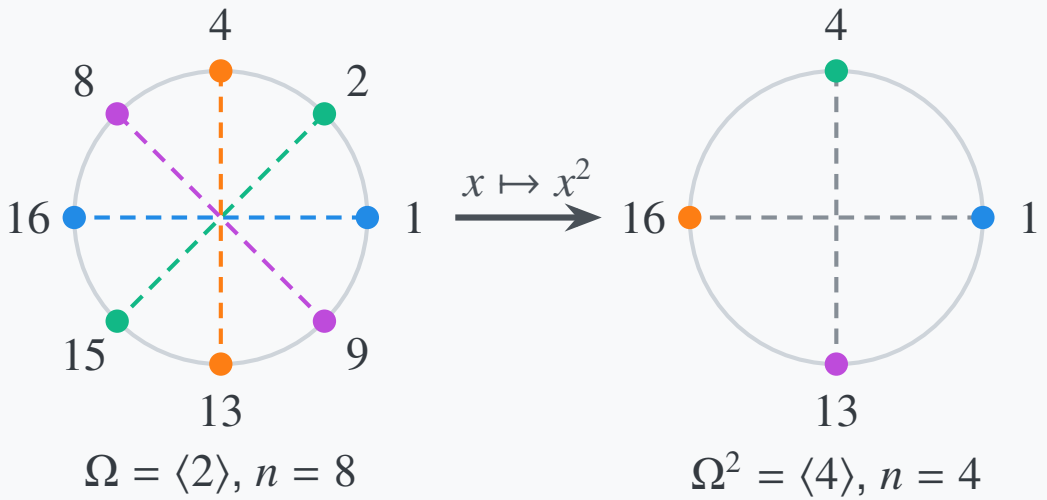


Figure: In $\mathbb{F}_{17}$: in $\Omega = \langle 2 \rangle$, every x has $-x$ on the opposite side (dashed), and both are mapped to the point of the same colour in Ω^2. Applying the map again gives $\Omega^4 = \{1, -1\}$.

Squaring Halves a Smooth Domain

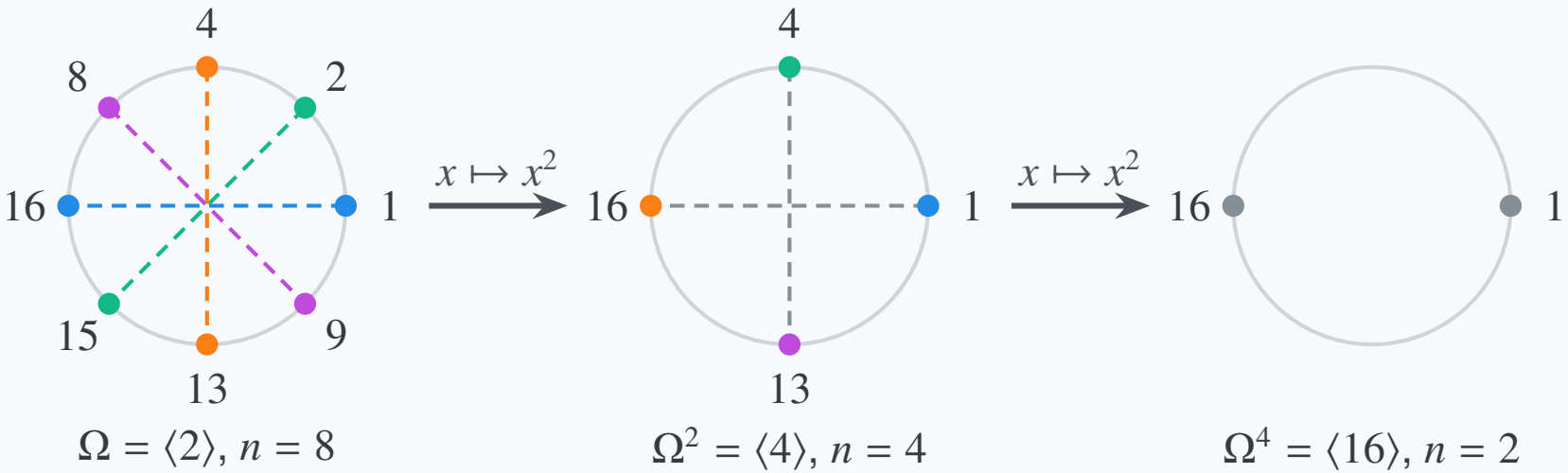


Figure: In $\mathbb{F}_{17}$: in $\Omega = \langle 2 \rangle$, every x has $-x$ on the opposite side (dashed), and both are mapped to the point of the same colour in Ω^2. Applying the map again gives $\Omega^4 = \{1, -1\}$.

2-way Folding

If $g = \bar{f}$, plugging x and $-x$ into $f(T) = f_E(T^2) + Tf_O(T^2)$ gives two linear equations:

$$f_E(x^2) = \frac{g(x) + g(-x)}{2}, \qquad f_O(x^2) = \frac{g(x) - g(-x)}{2x}.$$

2-way Folding

If $g = \bar{f}$, plugging x and $-x$ into $f(T) = f_E(T^2) + Tf_O(T^2)$ gives two linear equations:

$$f_E(x^2) = \frac{g(x) + g(-x)}{2}, \qquad f_O(x^2) = \frac{g(x) - g(-x)}{2x}.$$

The right-hand sides make sense for an **arbitrary** restriction g.

Definition (2-way folding)

For a restriction $g : \Omega \to \mathbb{F}$ and $r \in \mathbb{F}$, define $g_E, g_O : \Omega^2 \to \mathbb{F}$ by

$$g_E(x^2) := \frac{g(x) + g(-x)}{2}, \qquad g_O(x^2) := \frac{g(x) - g(-x)}{2x},$$

and the **fold** of g as $\mathrm{Fold}(g, r) := g_E + r \cdot g_O : \Omega^2 \to \mathbb{F}$.

Folding is Distance Preserving

Lemma

Let $C := \mathsf{RS}[\mathbb{F}, \Omega, k]$, $C' := \mathsf{RS}[\mathbb{F}, \Omega^2, k/2]$, and δ, ε_δ as in the proximity gap lemma. Then:

1. if $g = \overline{f} \in C$, then $\mathsf{Fold}(g, r) = \overline{f_E + r f_O} \in C'$ for all $r \in \mathbb{F}$;
2. if g is δ-far from C, then $\Pr_{r \leftarrow \$ \mathbb{F}}[\mathsf{Fold}(g, r) \text{ is } \delta\text{-close to } C'] \leq \varepsilon_\delta$.

Folding is Distance Preserving

Lemma

Let $C := \mathrm{RS}[\mathbb{F}, \Omega, k]$, $C' := \mathrm{RS}[\mathbb{F}, \Omega^2, k/2]$, and δ, ε_δ as in the proximity gap lemma. Then:

1. if $g = \overline{f} \in C$, then $\mathrm{Fold}(g, r) = \overline{f_E + r f_O} \in C'$ for all $r \in \mathbb{F}$;
2. if g is δ-far from C, then $\Pr_{r \leftarrow \$ \mathbb{F}}[\mathrm{Fold}(g, r) \text{ is } \delta\text{-close to } C'] \leq \varepsilon_\delta$.

Proof idea of (ii). $\mathrm{Fold}(g, r)$ is a random linear combination of g_E, g_O; C, C' have the same rate.

Folding is Distance Preserving

Lemma

Let $C := \mathrm{RS}[\mathbb{F}, \Omega, k]$, $C' := \mathrm{RS}[\mathbb{F}, \Omega^2, k/2]$, and δ, ε_δ as in the proximity gap lemma. Then:

1. if $g = \overline{f} \in C$, then $\mathrm{Fold}(g, r) = \overline{f_E + r f_O} \in C'$ for all $r \in \mathbb{F}$;
2. if g is δ-far from C, then $\Pr_{r \leftarrow \$ \mathbb{F}}[\mathrm{Fold}(g, r) \text{ is } \delta\text{-close to } C'] \leq \varepsilon_\delta$.

Proof idea of (ii). $\mathrm{Fold}(g, r)$ is a random linear combination of g_E, g_O; C, C' have the same rate.

- The theorem gives $S \subseteq \Omega^2$ and $u_E, u_O \in \mathbb{F}[T]^{<k/2}$ agreeing with g_E, g_O on S.

Folding is Distance Preserving

Lemma

Let $C := \mathrm{RS}[\mathbb{F}, \Omega, k]$, $C' := \mathrm{RS}[\mathbb{F}, \Omega^2, k/2]$, and δ, ε_δ as in the proximity gap lemma. Then:

1. if $g = \overline{f} \in C$, then $\mathrm{Fold}(g, r) = \overline{f_E + r f_O} \in C'$ for all $r \in \mathbb{F}$;
2. if g is δ-far from C, then $\Pr_{r \leftarrow \$ \mathbb{F}}[\mathrm{Fold}(g, r) \text{ is } \delta\text{-close to } C'] \leq \varepsilon_\delta$.

Proof idea of (ii). $\mathrm{Fold}(g, r)$ is a random linear combination of g_E, g_O; C, C' have the same rate.

- The theorem gives $S \subseteq \Omega^2$ and $u_E, u_O \in \mathbb{F}[T]^{<k/2}$ agreeing with g_E, g_O on S.
- Glue: $u(T) := u_E(T^2) + T \cdot u_O(T^2)$ agrees with g on the $2 \cdot \#S \geq (1 - \delta)n$ preimages of S: contradiction. □

Folding is Distance Preserving

Lemma

Let $C := \mathrm{RS}[\mathbb{F}, \Omega, k]$, $C' := \mathrm{RS}[\mathbb{F}, \Omega^2, k/2]$, and δ, ε_δ as in the proximity gap lemma. Then:

1. if $g = \overline{f} \in C$, then $\mathrm{Fold}(g, r) = \overline{f_E + r f_O} \in C'$ for all $r \in \mathbb{F}$;
2. if g is δ-far from C, then $\Pr_{r \leftarrow \$ \mathbb{F}}[\mathrm{Fold}(g, r) \text{ is } \delta\text{-close to } C'] \leq \varepsilon_\delta$.

Proof idea of (ii). $\mathrm{Fold}(g, r)$ is a random linear combination of g_E, g_O; C, C' have the same rate.

- The theorem gives $S \subseteq \Omega^2$ and $u_E, u_O \in \mathbb{F}[T]^{<k/2}$ agreeing with g_E, g_O on S.
- Glue: $u(T) := u_E(T^2) + T \cdot u_O(T^2)$ agrees with g on the $2 \cdot \#S \geq (1 - \delta)n$ preimages of S: contradiction. □

Remark

We need the **strong** form: the **same** set S for g_E and g_O.

Attempt #3: Putting Everything Together

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

1: **Commit.** $\mathcal{P}$ sends oracles $g_1, \ldots, g_m : \Omega \to \mathbb{F}$; honestly, $g_j := \overline{f_j}$.

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

1: **Commit.** $\mathcal{P}$ sends oracles $g_1, \ldots, g_m : \Omega \to \mathbb{F}$; honestly, $g_j := \overline{f_j}$.

2: **Bind.** $\mathcal{V}$ samples $r_0 \leftarrow\!\!\$\ \mathbb{F} \setminus \Omega$, $\mathcal{P}$ replies with $y_{j,0}$ $(= f_j(r_0))$ for every $j \in [m]$.

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

1: **Commit.** $\mathcal{P}$ sends oracles $g_1, \ldots, g_m : \Omega \to \mathbb{F}$; honestly, $g_j := \overline{f_j}$.

2: **Bind.** $\mathcal{V}$ samples $r_0 \leftarrow\!\!\$\ \mathbb{F} \setminus \Omega$, $\mathcal{P}$ replies with $y_{j,0}$ $(= f_j(r_0))$ for every $j \in [m]$.

3: **Run the Poly-IOP.** When asked for $f_j(r_i)$, $\mathcal{P}$ replies with $y_{j,i}$, and $\mathcal{V}$ simply uses it.

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

1: **Commit.** $\mathcal{P}$ sends oracles $g_1, \ldots, g_m : \Omega \to \mathbb{F}$; honestly, $g_j := \overline{f_j}$.
2: **Bind.** $\mathcal{V}$ samples $r_0 \leftarrow\!\!\$\ \mathbb{F} \setminus \Omega$, $\mathcal{P}$ replies with $y_{j,0}$ $(= f_j(r_0))$ for every $j \in [m]$.
3: **Run the Poly-IOP.** When asked for $f_j(r_i)$, $\mathcal{P}$ replies with $y_{j,i}$, and $\mathcal{V}$ simply uses it.
4: **Quotient.** $v(T) := \prod_{i=0}^{t}(T - r_i)$, $\iota_j \in \mathbb{F}[T]^{\leq t}$ with $\iota_j(r_i) = y_{j,i}$, and

$$\mathsf{q}_j := \frac{g_j - \iota_j}{v} : \Omega \to \mathbb{F}, \quad j \in [m].$$

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

1: **Commit.** $\mathcal{P}$ sends oracles $g_1, \ldots, g_m : \Omega \to \mathbb{F}$; honestly, $g_j := \overline{f_j}$.

2: **Bind.** $\mathcal{V}$ samples $r_0 \leftarrow\!\$\ \mathbb{F} \setminus \Omega$, $\mathcal{P}$ replies with $y_{j,0}$ $(= f_j(r_0))$ for every $j \in [m]$.

3: **Run the Poly-IOP.** When asked for $f_j(r_i)$, $\mathcal{P}$ replies with $y_{j,i}$, and $\mathcal{V}$ simply uses it.

4: **Quotient.** $\nu(T) := \prod_{i=0}^{t}(T - r_i)$, $\iota_j \in \mathbb{F}[T]^{\leq t}$ with $\iota_j(r_i) = y_{j,i}$, and

$$\mathsf{q}_j := \frac{g_j - \iota_j}{\nu} : \Omega \to \mathbb{F}, \quad j \in [m].$$

5: **Batch.** $\mathcal{V}$ samples $\alpha \leftarrow\!\$\ \mathbb{F}$; both define $h := \sum_{j \in [m]} \alpha^j \mathsf{q}_j$.

Attempt #3: The Full Compilation

The Poly-IOP prover sends $f_1, \ldots, f_m \in \mathbb{F}[T]^{<k}$; the Poly-IOP verifier wants all of them at $r_1, \ldots, r_t \in \mathbb{F} \setminus \Omega$.

1: **Commit.** $\mathcal{P}$ sends oracles $g_1, \ldots, g_m : \Omega \to \mathbb{F}$; honestly, $g_j := \overline{f_j}$.
2: **Bind.** $\mathcal{V}$ samples $r_0 \leftarrow\!\$\ \mathbb{F} \setminus \Omega$, $\mathcal{P}$ replies with $y_{j,0}$ $(= f_j(r_0))$ for every $j \in [m]$.
3: **Run the Poly-IOP.** When asked for $f_j(r_i)$, $\mathcal{P}$ replies with $y_{j,i}$, and $\mathcal{V}$ simply uses it.
4: **Quotient.** $\nu(T) := \prod_{i=0}^{t}(T - r_i)$, $\iota_j \in \mathbb{F}[T]^{\leq t}$ with $\iota_j(r_i) = y_{j,i}$, and

$$\mathsf{q}_j := \frac{g_j - \iota_j}{\nu} : \Omega \to \mathbb{F}, \quad j \in [m].$$

5: **Batch.** $\mathcal{V}$ samples $\alpha \leftarrow\!\$\ \mathbb{F}$; both define $h := \sum_{j\in[m]} \alpha^j \mathsf{q}_j$.
6: **Test.** Run a single δ-RS-IOPP for h against $\mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$.

Attempt #3

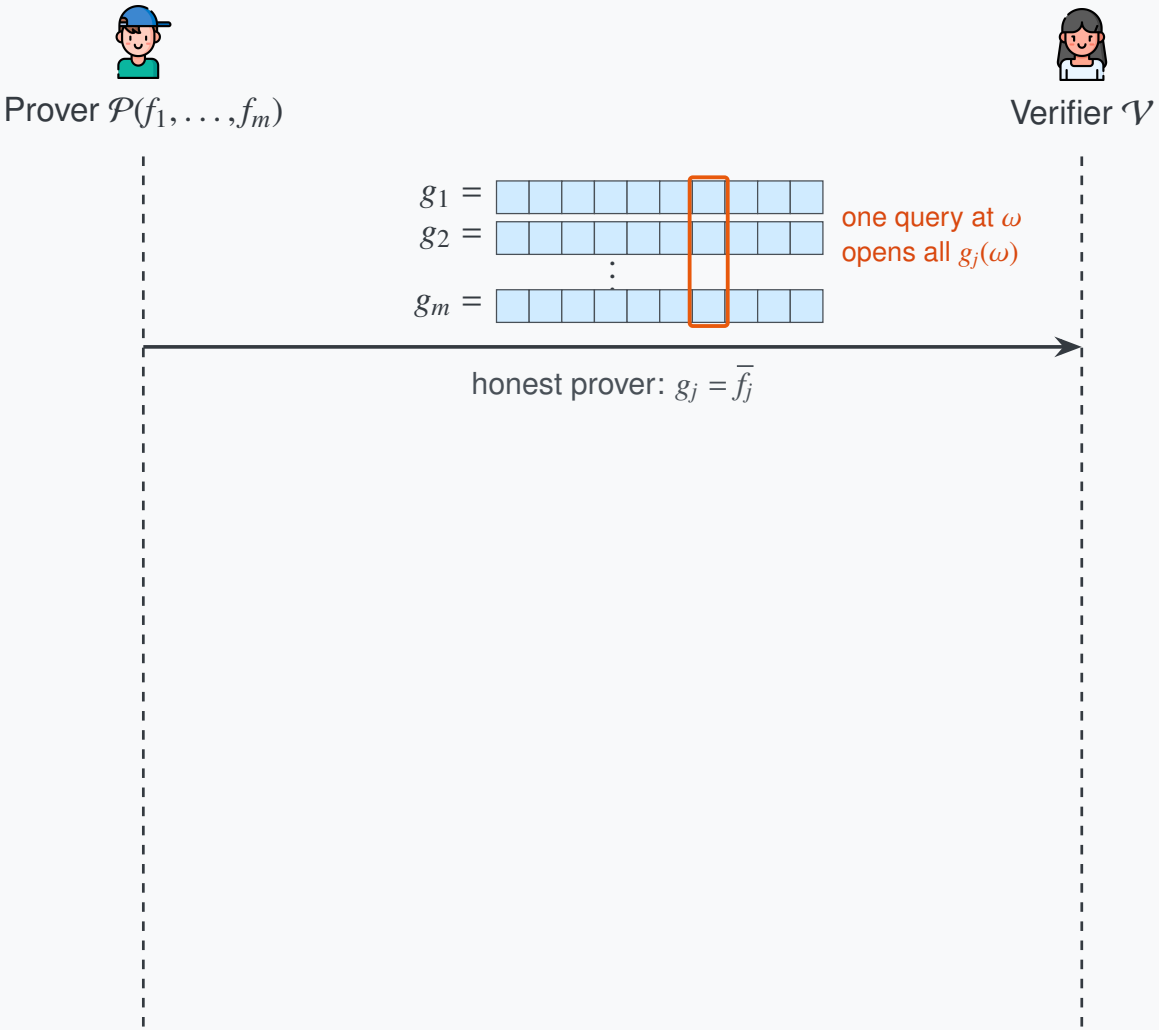


Figure: Commit, bind, answer, quotient, batch with α (highlighted), and run only one RS-IOPP.

Attempt #3

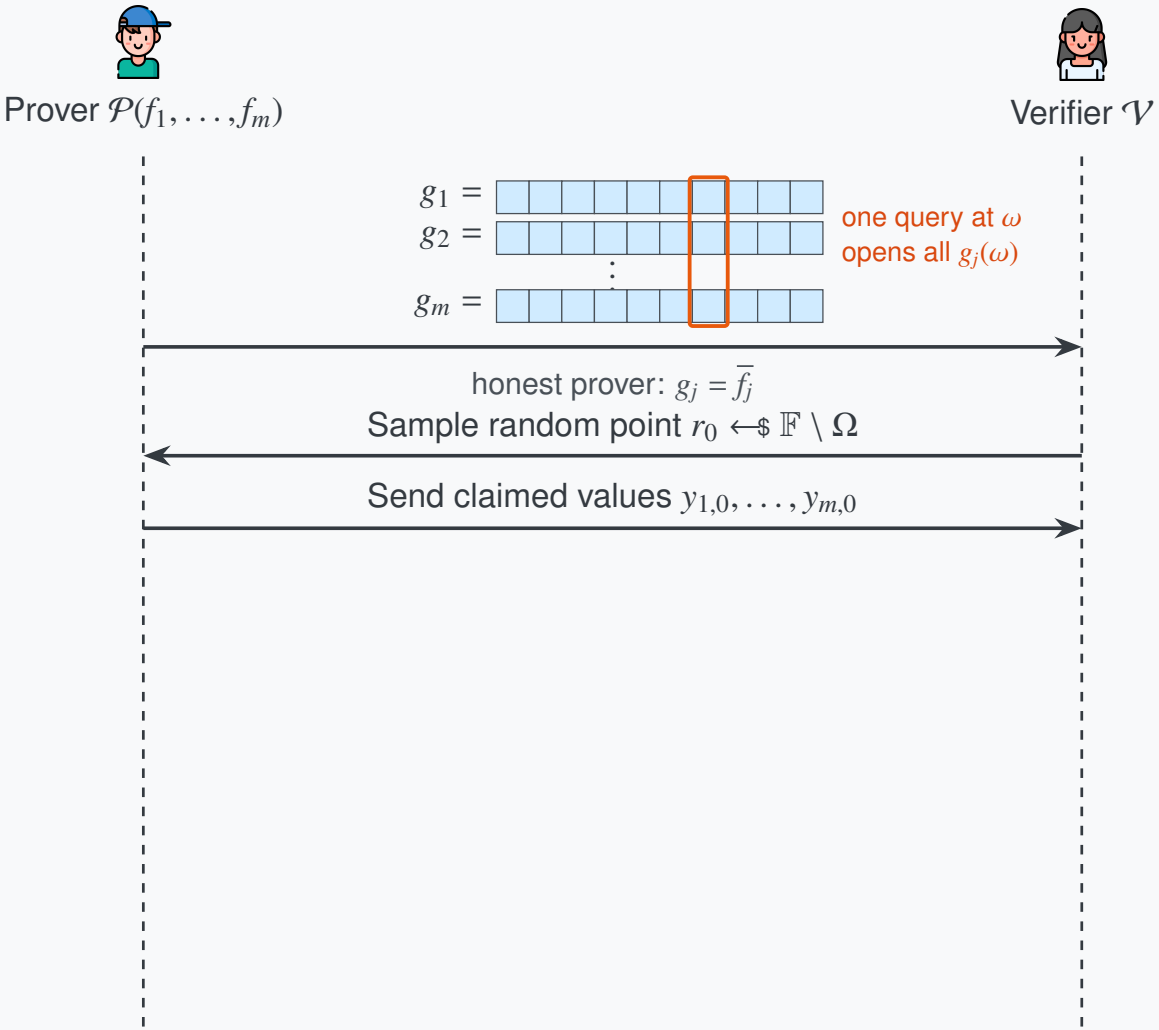


Figure: Commit, bind, answer, quotient, batch with α (highlighted), and run only one RS-IOPP.

Attempt #3

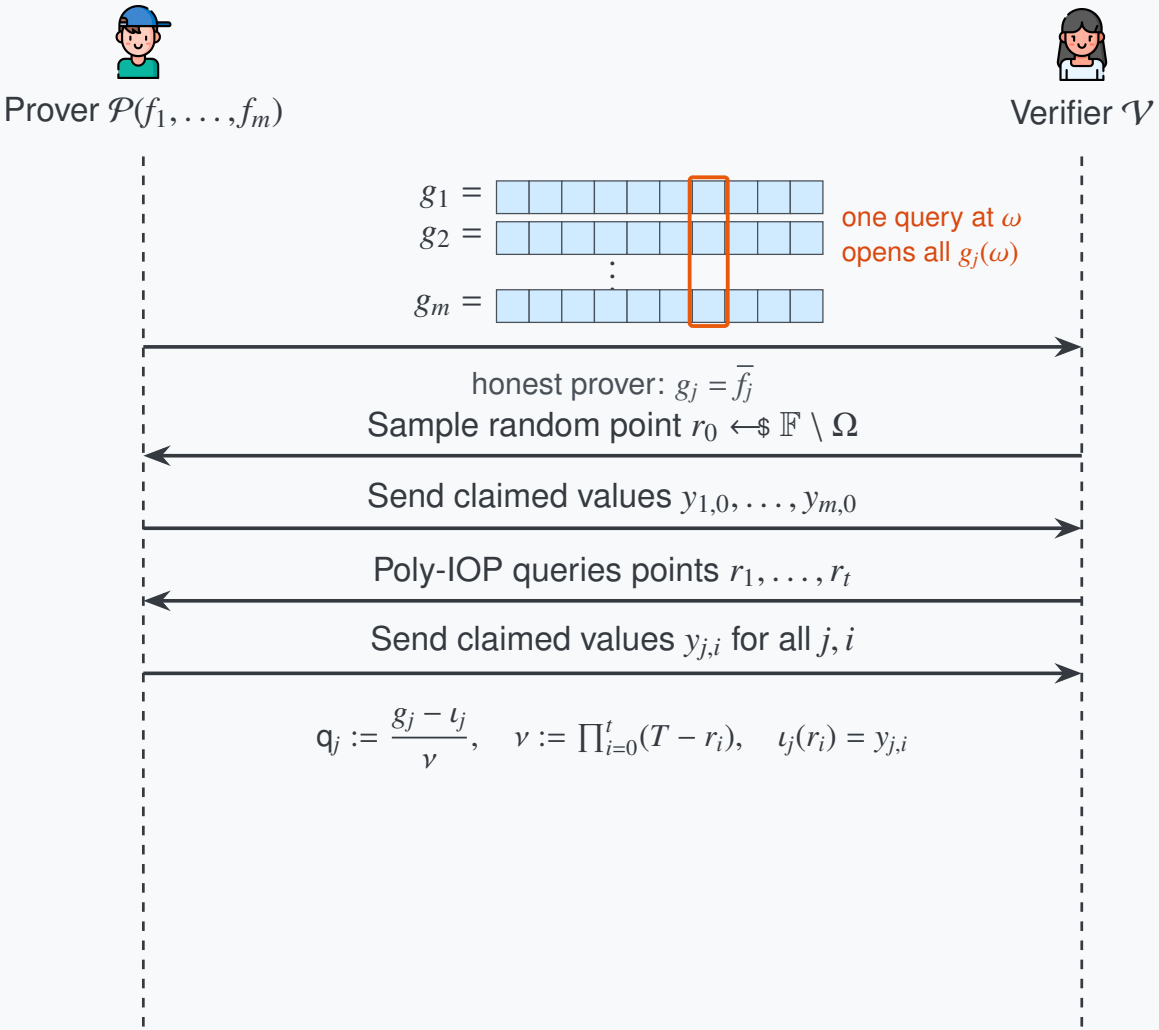


Figure: Commit, bind, answer, quotient, batch with α (highlighted), and run only one RS-IOPP.

Attempt #3

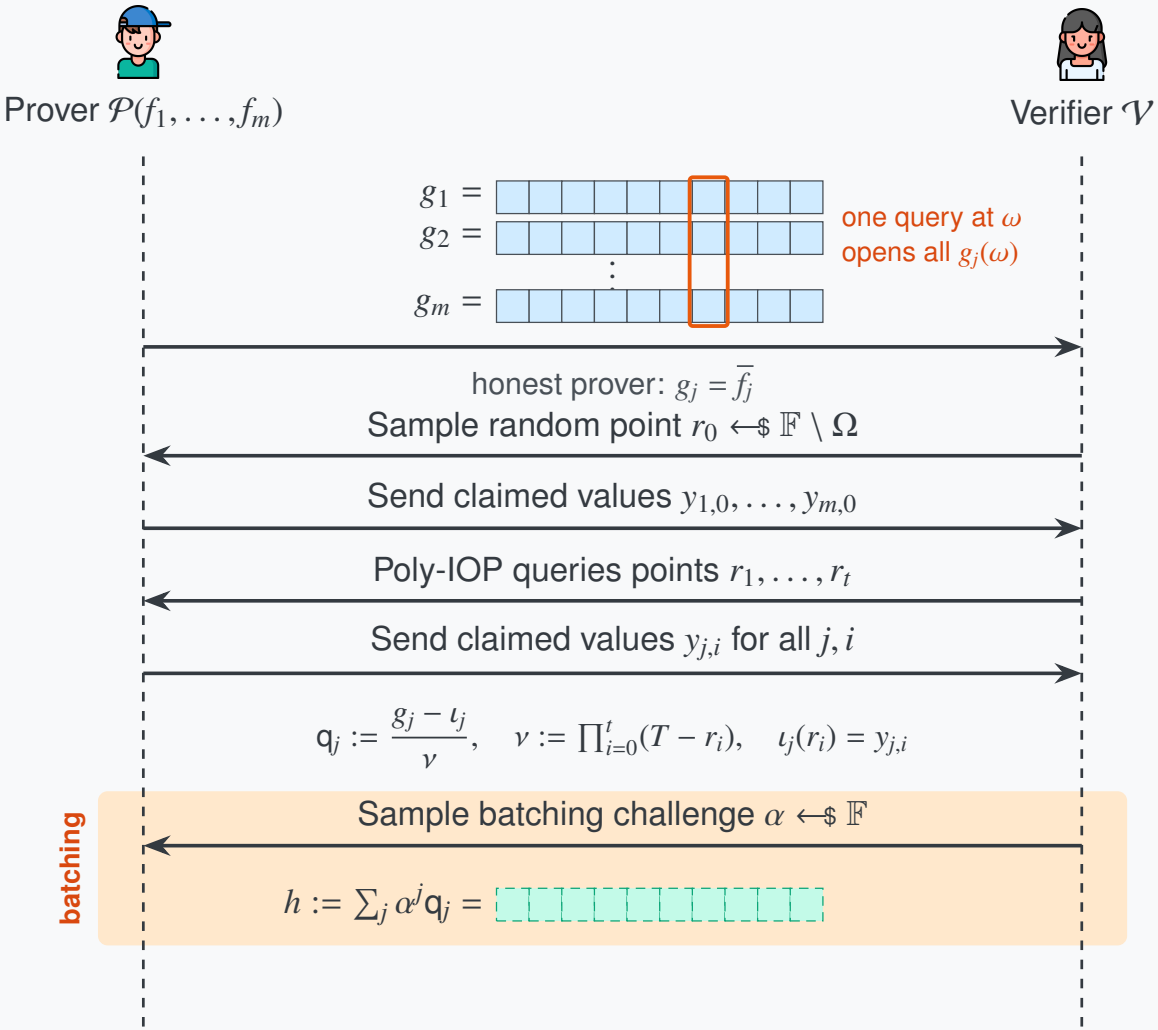


Figure: Commit, bind, answer, quotient, batch with α (highlighted), and run only one RS-IOPP.

Attempt #3

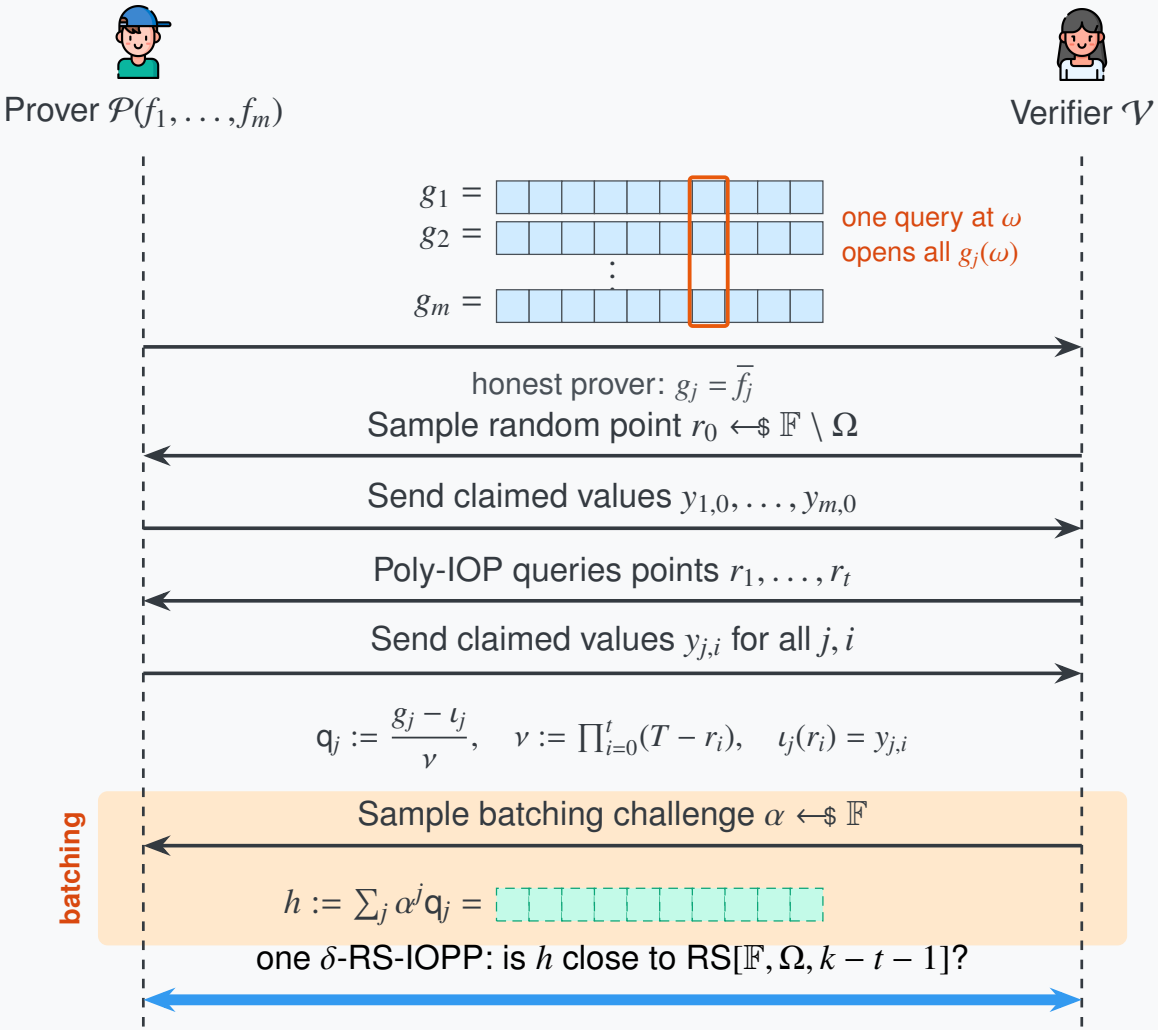


Figure: Commit, bind, answer, quotient, batch with α (highlighted), and run only one RS-IOPP.

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Completeness. Every $\mathsf{q}_j \in \mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$, hence so is h.

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Completeness. Every $\mathsf{q}_j \in \mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$, hence so is h.

Soundness. Suppose $\mathcal{V}$ accepts with non-negligible probability. Then:

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Completeness. Every $\mathsf{q}_j \in \mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$, hence so is h.

Soundness. Suppose $\mathcal{V}$ accepts with non-negligible probability. Then:

1: h is δ-close to the code, so by proximity gaps every q_j is δ-close to $\mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$ (except w.p. ε_δ over α);

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Completeness. Every $\mathsf{q}_j \in \mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$, hence so is h.

Soundness. Suppose $\mathcal{V}$ accepts with non-negligible probability. Then:

1: h is δ-close to the code, so by proximity gaps every q_j is δ-close to $\mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$ (except w.p. ε_δ over α);

2: by multiple quotienting, each g_j is δ-close to some $\overline{f'_j}$ with $f'_j(r_i) = y_{j,i}$ for **all** i;

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Completeness. Every $\mathsf{q}_j \in \mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$, hence so is h.

Soundness. Suppose $\mathcal{V}$ accepts with non-negligible probability. Then:

1: h is δ-close to the code, so by proximity gaps every q_j is δ-close to $\mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$ (except w.p. ε_δ over α);

2: by multiple quotienting, each g_j is δ-close to some $\overline{f'_j}$ with $f'_j(r_i) = y_{j,i}$ for **all** i;

3: $\overline{f'_j}$ is the only codeword in $\mathsf{List}[g_j, k, \delta]$ through $(r_0, y_{j,0})$, except w.p. $\Pr[\mathsf{Bad}]$ (union bound over j).

Attempt #3: Why it Works

Virtual oracles. No oracles for q_j or h: $\mathcal{V}$ queries $g_1(\omega), \ldots, g_m(\omega)$ and computes $h(\omega)$.

Completeness. Every $\mathsf{q}_j \in \mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$, hence so is h.

Soundness. Suppose $\mathcal{V}$ accepts with non-negligible probability. Then:

1: h is δ-close to the code, so by proximity gaps every q_j is δ-close to $\mathsf{RS}[\mathbb{F}, \Omega, k - t - 1]$ (except w.p. ε_δ over α);

2: by multiple quotienting, each g_j is δ-close to some $\overline{f'_j}$ with $f'_j(r_i) = y_{j,i}$ for **all** i;

3: $\overline{f'_j}$ is the only codeword in $\mathsf{List}[g_j, k, \delta]$ through $(r_0, y_{j,0})$, except w.p. $\Pr[\mathsf{Bad}]$ (union bound over j).

All answers come from one fixed f'_j per oracle: exactly what a polynomial commitment must guarantee.

Degree Correction

Remark

If different polynomials are queried at different numbers of points, the quotients q_j have different degree bounds and cannot be added directly.

Degree Correction

Remark

If different polynomials are queried at different numbers of points, the quotients q_j have different degree bounds and cannot be added directly.

Degree correction

Before batching, each q_j is multiplied by a power of T (and combined with fresh random coefficients), so that all of them are tested against the same code.

Degree Correction

Remark

If different polynomials are queried at different numbers of points, the quotients q_j have different degree bounds and cannot be added directly.

Degree correction

Before batching, each q_j is multiplied by a power of T (and combined with fresh random coefficients), so that all of them are tested against the same code.

Question #3

Why do we sample r_0 from $\mathbb{F} \setminus \Omega$ and not from Ω? What would go wrong with the quotient q_j otherwise?

Summary

- An **IOPP** gives the verifier only oracle access to the input $\mathbb{y}$; we can only ask it to reject inputs that are δ**-far** from valid ones.

Summary

- An **IOPP** gives the verifier only oracle access to the input $\mathbb{y}$; we can only ask it to reject inputs that are δ**-far** from valid ones.
- A δ**-RS-IOPP** tests proximity of $g : \Omega \to \mathbb{F}$ to RS$[\mathbb{F}, \Omega, k]$.

Summary

- An **IOPP** gives the verifier only oracle access to the input $\mathbb{y}$; we can only ask it to reject inputs that are δ**-far** from valid ones.
- A δ**-RS-IOPP** tests proximity of $g : \Omega \to \mathbb{F}$ to RS$[\mathbb{F}, \Omega, k]$.
- **Quotienting** turns the claim $f(r) = y$ into proximity of $(g - y)/(T - r)$ to a smaller RS code.

Summary

- An **IOPP** gives the verifier only oracle access to the input $\mathbb{y}$; we can only ask it to reject inputs that are δ**-far** from valid ones.
- A δ**-RS-IOPP** tests proximity of $g : \Omega \to \mathbb{F}$ to RS$[\mathbb{F}, \Omega, k]$.
- **Quotienting** turns the claim $f(r) = y$ into proximity of $(g - y)/(T - r)$ to a smaller RS code.
- **Out-of-domain sampling** at $r_0 \leftarrow\$ \mathbb{F} \setminus \Omega$ binds the prover to a single codeword of the list.

Summary

- An **IOPP** gives the verifier only oracle access to the input $\mathbb{y}$; we can only ask it to reject inputs that are δ**-far** from valid ones.
- A δ**-RS-IOPP** tests proximity of $g : \Omega \to \mathbb{F}$ to RS$[\mathbb{F}, \Omega, k]$.
- **Quotienting** turns the claim $f(r) = y$ into proximity of $(g - y)/(T - r)$ to a smaller RS code.
- **Out-of-domain sampling** at $r_0 \leftarrow\$ \mathbb{F} \setminus \Omega$ binds the prover to a single codeword of the list.
- **Proximity gaps**: random linear combinations and **folding** are distance preserving, so all claims are batched into one RS-IOPP.

Summary

- An **IOPP** gives the verifier only oracle access to the input $\mathbb{y}$; we can only ask it to reject inputs that are δ**-far** from valid ones.
- A δ**-RS-IOPP** tests proximity of $g : \Omega \to \mathbb{F}$ to RS$[\mathbb{F}, \Omega, k]$.
- **Quotienting** turns the claim $f(r) = y$ into proximity of $(g - y)/(T - r)$ to a smaller RS code.
- **Out-of-domain sampling** at $r_0 \leftarrow\$ \mathbb{F} \setminus \Omega$ binds the prover to a single codeword of the list.
- **Proximity gaps**: random linear combinations and **folding** are distance preserving, so all claims are batched into one RS-IOPP.
- Next: building the RS-IOPP itself (FRI) and the STARK.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream